KELOMPOK 3 · PEMROGRAMAN BERORIENTASI OBJEK

Penjelasan Project & Pembagian Materi Presentasi

Sistem Manajemen Data Pasien Rumah Sakit

Penerapan Class & Object, Enkapsulasi, dan Inheritance pada Python dengan data nyata dari file hospital data analysis.csv

984	10	4	9
data pasien nyata	kolom data	class + 1 program utama	pembicara

Pembagian Materi Anggota Kelompok 3

No	Nama	Bagian yang Dijelaskan	Durasi
1	**Riswan**	Pembukaan & Gambaran Umum Project	± 3 menit
2	**Yus Aidil**	Dataset & Aturan Data	± 3 menit
3	**Fahmi**	Konsep Class & Object (class Orang)	± 3,5 menit
4	**Diva Nurislamia Agfa**	Enkapsulasi: Atribut Private, Getter & Setter	± 3,5 menit
5	**Rosa Ramadani**	Inheritance (1): Pasien Mewarisi Orang	± 3 menit
6	**Sofiang**	Inheritance (2): Pewarisan Bertingkat & Polimorfisme	± 2,5 menit
7	**Revalinda**	Komposisi & Validasi: class RekamMedis	± 3,5 menit
8	**Andi Rezky Nabila**	Program Utama (main.py): Membaca CSV & Menu Interaktif	± 3,5 menit
9	**Alfira Yunita Syam**	Hasil Output, Demo Menu & Kesimpulan	± 4 menit

Cara memakai dokumen ini

- Setiap anggota membaca **bagiannya sendiri**: poin yang dijelaskan, kode yang ditunjukkan, naskah, dan persiapan tanya jawab.
- Nomor baris pada potongan kode **sama persis** dengan file project, jadi mudah ditunjukkan di layar. Di bawah setiap potongan kode ada tabel **fungsi tiap baris**.
- Naskah lengkap per slide ada di **Naskah Presentasi Kelompok 3.pdf**. Pahami alurnya, sampaikan dengan **bahasa sendiri**, dan akhiri dengan menyebut nama pembicara berikutnya.
- Output program memakai format angka Inggris (koma = pemisah ribuan, titik = desimal). Saat dibacakan, pakai format Indonesia, misalnya `8,367.48` dibaca “delapan ribu tiga ratus enam puluh tujuh koma empat delapan”.
- Semua output di dokumen ini diambil dari program yang **benar-benar dijalankan** (termasuk sesi demo menu), bukan contoh karangan.

November 2026

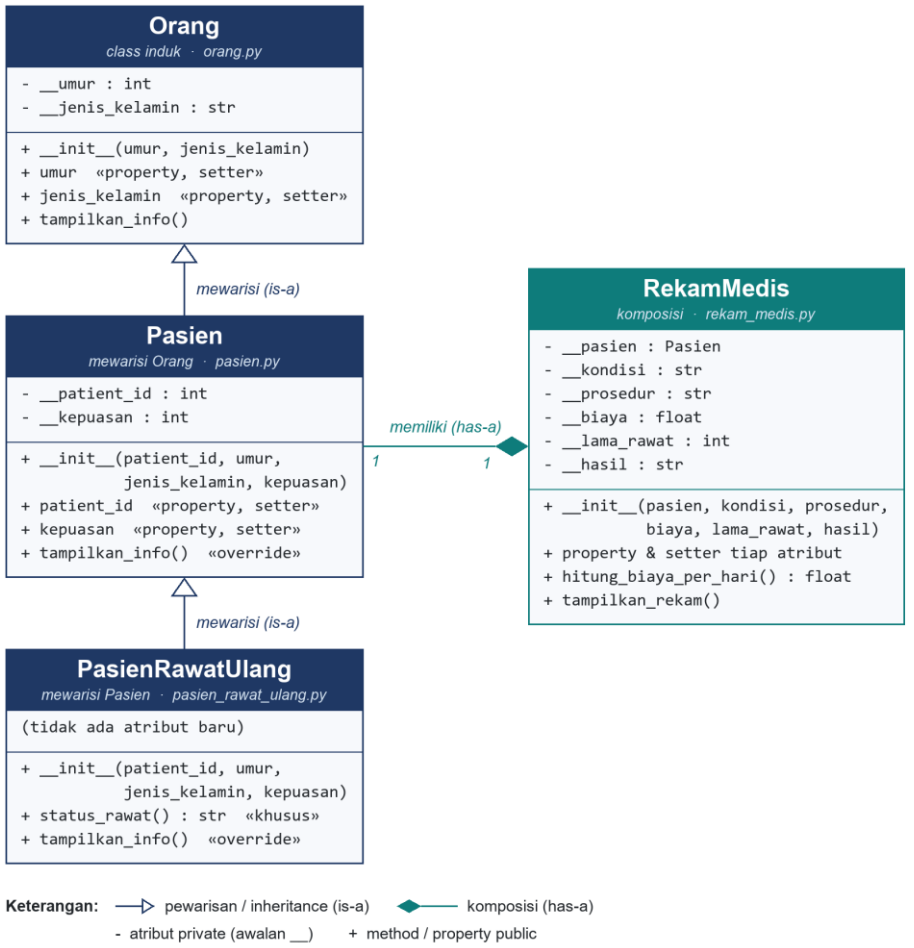
GAMBARAN UMUM PROJECT

Program **Sistem Manajemen Data Pasien Rumah Sakit** membaca 984 data pasien asli dari file CSV, mengubah setiap baris menjadi object, lalu menyimpannya di dalam list. Baris yang tidak valid dilewati, nomornya dicatat, dan jumlahnya ditampilkan. Setelah itu program menampilkan 5 rekam medis pertama, ringkasan data, dan demo validasi data, lalu membuka **menu interaktif** untuk melihat 10 data pertama, mencari pasien berdasarkan Patient_ID, dan menambah pasien baru. Kode dibagi menjadi 4 file class dan 1 file program utama, dan hanya memakai library bawaan Python.

Hasil Ringkasan dari Data Asli

Ringkasan	Hasil (sesuai output program)
Total pasien	984
Jumlah pasien rawat ulang	264 (26.83%)
Rata-rata biaya	8,367.48
Rata-rata lama rawat	37.66 hari
Kondisi paling sering	Fractured Leg, Heart Attack (masing-masing 67 pasien)

Diagram Class



Gambar: diagram class Sistem Manajemen Data Pasien Rumah Sakit

BAGIAN 1 | RISWAN

Pembukaan & Gambaran Umum Project

Durasi: ± 3 menit • **Konsep:** Pengantar • **Fokus:** Struktur folder & cara menjalankan

Poin yang Dijelaskan

- Membuka presentasi, memperkenalkan **Kelompok 3** dan judul project: **Sistem Manajemen Data Pasien Rumah Sakit**.
- Latar belakang singkat: rumah sakit mengelola banyak data pasien (identitas, kondisi, biaya, lama rawat), sehingga perlu program yang rapi dan datanya aman. OOP cocok karena setiap pasien bisa dimodelkan sebagai **object**.
- Tujuan project: menerapkan tiga konsep OOP, yaitu **Class & Object**, **Enkapsulasi**, dan **Inheritance**, memakai **data nyata** (bukan data karangan) dari file CSV berisi 984 pasien.
- Alur kerja program: baca CSV → ubah setiap baris menjadi object (baris tidak valid dilewati dan nomornya dicatat) → simpan di list → tampilkan 5 rekam medis pertama, ringkasan data, dan demo validasi → **menu interaktif**: lihat 10 data, cari pasien berdasarkan Patient_ID, tambah pasien baru, keluar.
- Struktur file: **satu class per file** (`orang.py`, `pasien.py`, `pasien_rawat_ulang.py`, `rekam_medis.py`) ditambah `main.py`, semuanya memakai penamaan *snake_case*.
- Program hanya memakai library bawaan Python (`csv` dan `os`), dan cukup dijalankan dengan perintah `python main.py`.
- Menyebutkan urutan pembicara, lalu menyerahkan ke **Yus Aidil**.

Kode & Tampilan yang Ditunjukkan

Struktur folder project

```
Tugas Kelompok/
├── hospital data analysis.csv   → data asli (984 pasien, 10 kolom)
├── orang.py                     → class Orang (class induk)
├── pasien.py                    → class Pasien (mewarisi Orang)
├── pasien_rawat_ulang.py        → class PasienRawatUlang (mewarisi Pasien)
├── rekam_medis.py               → class RekamMedis (komposisi)
└── main.py                      → program utama + menu interaktif
```

Perintah & awal output

```
> python main.py
============================================================
SISTEM MANAJEMEN DATA PASIEN RUMAH SAKIT
============================================================
File data         : hospital data analysis.csv
Object RekamMedis : 984 object tersimpan di list
Baris dilewati    : 0
```

Naskah Penyampaian

Sama dengan **Naskah Presentasi Kelompok 3.pdf** dan catatan pembicara PPT. Versi lengkap dengan tanda KLIK dan tunjuk ada di dokumen naskah.

> **Slide 1** *Selamat pagi Bapak/Ibu dosen dan teman-teman semua. Kami dari Kelompok 3 akan mempresentasikan project berjudul Sistem Manajemen Data Pasien Rumah Sakit, yang kami buat dengan bahasa Python untuk menerapkan konsep pemrograman berorientasi objek.*

Slide 2 *Presentasi kami dibagi menjadi sembilan bagian, sesuai jumlah anggota. Saya, Riswan, membuka presentasi. Lalu Yus Aidil menjelaskan dataset, Fahmi tentang class dan object, Diva tentang enkapsulasi, Rosa dan Sofiang tentang inheritance, Revalinda tentang komposisi dan validasi data medis, Andi tentang program utama dan menu interaktif, dan Alfira menunjukkan hasil, demo menu, serta kesimpulan.*

Slide 3 *Kita mulai dari bagian pertama, yaitu gambaran umum project.*

Slide 4 *Situasinya, rumah sakit mencatat banyak data pasien setiap hari, dari identitas sampai biaya. Masalahnya, data seperti ini mudah berantakan dan rawan salah input, misalnya umur negatif atau kepuasan tujuh, padahal skalanya hanya satu sampai lima. Jadi pertanyaannya, bagaimana mengelola data ini secara terstruktur dan aman? Jawaban kami adalah program Python berbasis OOP: setiap pasien menjadi object yang datanya dijaga. Kami memakai data nyata 984 pasien dan menerapkan tiga konsep: class dan object, enkapsulasi, serta inheritance.*

Slide 5 *Alurnya: program membaca file CSV dan mengubah setiap baris menjadi object; baris yang datanya salah dilewati dan nomornya dicatat. Object disimpan di list, lalu program menampilkan lima rekam medis pertama, ringkasan, dan demo validasi. Terakhir ada menu interaktif untuk melihat, mencari, dan menambah pasien. Kodenya dibagi menjadi lima file, satu class satu file, ditambah main.py sebagai program utama. Dengan pembagian ini, setiap file punya satu tugas yang jelas dan mudah dicari. Perintah python main.py fungsinya menjalankan program utama. Program hanya memakai library bawaan, jadi tidak perlu instalasi tambahan. Selanjutnya, Yus Aidil akan menjelaskan dataset yang kami gunakan.*

Persiapan Tanya Jawab

T **Kenapa memakai OOP, bukan program biasa (prosedural)?**

J Karena data pasien punya ciri (atribut) dan perilaku (method) yang jelas. Dengan OOP, setiap pasien menjadi object yang rapi, datanya terlindungi lewat enkapsulasi, dan kodenya bisa dipakai ulang lewat inheritance.

T **Kenapa satu class satu file?**

J Supaya kode mudah dicari, dibaca, dan dikembangkan. Setiap file hanya punya satu tanggung jawab.

T **Kenapa hanya memakai library bawaan?**

J Supaya program bisa langsung dijalankan di komputer mana pun yang sudah terpasang Python, tanpa instalasi tambahan.

BAGIAN 2 | YUS AIDIL

Dataset & Aturan Data

Durasi: ± 3 menit • **Konsep:** Data nyata • **Fokus:** hospital data analysis.csv

Poin yang Dijelaskan

- Data berasal dari file **hospital data analysis.csv**: **984 baris** data pasien dengan **10 kolom**. Sudah dicek: tidak ada nilai kosong.
- Kolom data: `Patient_ID`, Age, `Gender`, `Condition`, `Procedure`, `Cost`, `Length_of_Stay`, `Readmission`, `Outcome`, `Satisfaction`.
- Aturan nilai valid: Gender = Male/Female, Readmission = Yes/No, Outcome = Recovered/Stable, Satisfaction = 1 sampai 5, Age dan Cost bilangan positif, Length_of_Stay bilangan bulat lebih dari 0, Condition dan Procedure tidak boleh kosong.
- Gambaran isi data: 524 Female dan 460 Male; 264 pasien rawat ulang (Yes) dan 720 tidak; 591 Recovered dan 393 Stable; 15 jenis kondisi; umur 25–78 tahun; lama rawat 1–76 hari; Patient_ID unik 1–1000 (16 nomor tidak dipakai, misalnya 60).
- Aturan penting project: **tidak ada atribut tambahan di luar kolom data**. Setiap kolom dipetakan ke atribut class (lihat tabel di bawah).
- Kolom **Readmission** tidak dijadikan atribut, tetapi diwakili oleh jenis class: Yes → `PasienRawatUlang`, No → `Pasien`.

Kode & Tampilan yang Ditunjukkan

Isi awal file CSV (header + 2 baris pertama)

```
Patient_ID,Age,Gender,Condition,Procedure,Cost,Length_of_Stay,Readmission,Outcome,Satisfaction
1,45,Female,Heart Disease,Angioplasty,15000,5,No,Recovered,4
2,60,Male,Diabetes,Insulin Therapy,2000,3,Yes,Stable,3
```

Pemetaan kolom CSV ke atribut dan class

Kolom CSV	Atribut	Class	Aturan validasi (setter)
`Patient_ID`	`patient_id`	Pasien	bilangan bulat positif
`Age`	`umur`	Orang	bilangan bulat positif
`Gender`	`jenis_kelamin`	Orang	Male / Female
`Condition`	`kondisi`	RekamMedis	tidak boleh kosong
`Procedure`	`prosedur`	RekamMedis	tidak boleh kosong
`Cost`	`biaya`	RekamMedis	bilangan positif
`Length_of_Stay`	`lama_rawat`	RekamMedis	bilangan bulat > 0
`Readmission`	`(jenis class)`	Pasien / PasienRawatUlang	Yes / No (dicek di main.py)
`Outcome`	`hasil`	RekamMedis	Recovered / Stable
`Satisfaction`	`kepuasan`	Pasien	bilangan bulat 1–5

Naskah Penyampaian

Sama dengan **Naskah Presentasi Kelompok 3.pdf** dan catatan pembicara PPT. Versi lengkap dengan tanda KLIK dan tunjuk ada di dokumen naskah.

> **Slide 6** *Terima kasih, Riswan. Saya Yus Aidil, akan menjelaskan dataset dan aturan data yang kami pakai.*

Slide 7 *Data kami berasal dari file hospital data analysis.csv. Isinya 984 baris data pasien dengan 10 kolom, dan setelah kami periksa, tidak ada satu pun nilai kosong. Jenis kondisi penyakitnya ada 15. Baris pertama file disebut header, berisi nama kolom: ID pasien, umur, jenis kelamin, kondisi, prosedur, biaya, lama rawat, status rawat ulang, hasil perawatan, dan kepuasan. Baris kedua dan seterusnya adalah data pasien, dipisahkan tanda koma; misalnya pasien nomor satu berumur empat puluh lima tahun dengan penyakit jantung. Umur pasien di data ini antara 25 sampai 78 tahun, dan lama rawatnya antara 1 sampai 76 hari. Sebarannya: 524 pasien perempuan dan 460 laki-laki. Sebanyak 264 pasien dirawat ulang, sedangkan 720 tidak. Untuk hasil perawatan, 591 pasien Recovered atau sembuh, dan 393 berstatus Stable. Kebersihan data ini penting, karena program kami memeriksa setiap nilai; baris yang melanggar aturan akan dilewati.*

Slide 8 *Setiap kolom kami jadikan atribut di class, tanpa menambah atribut lain. Misalnya kolom Age menjadi atribut umur di class Orang, Satisfaction menjadi kepuasan di class Pasien, dan Cost menjadi biaya di class RekamMedis. Jadi data identitas ada di Orang dan Pasien, sedangkan data perawatan ada di RekamMedis, sehingga struktur class mengikuti struktur data aslinya. Di kolom paling kanan ada aturan nilainya: jenis kelamin hanya Male atau Female, hasil hanya Recovered atau Stable, kepuasan satu sampai lima, umur dan biaya harus positif, lama rawat bilangan bulat lebih dari nol, serta kondisi dan prosedur tidak boleh kosong. Aturan ini nanti dijaga oleh setter. Pemetaan ini juga memudahkan saat membaca CSV, karena setiap kolom langsung jelas masuk ke atribut mana. Khusus kolom Readmission, nilainya tidak disimpan sebagai atribut, tetapi diwakili oleh jenis class: Yes menjadi PasienRawatUlang, No menjadi Pasien. Selanjutnya, Fahmi akan menjelaskan konsep class dan object.*

Persiapan Tanya Jawab

T **Data ini data nyata atau buatan sendiri?**

J Data nyata dari file CSV, bukan data karangan. Program membaca file tersebut secara langsung setiap kali dijalankan.

T **Bagaimana kalau ada data yang tidak sesuai aturan?**

J Baris itu dilewati dengan peringatan, nomor barisnya dicatat, dan jumlah baris yang dilewati ditampilkan setelah data dimuat. Program tetap berjalan. Pada data asli, 0 baris dilewati.

T **Kenapa Readmission tidak dijadikan atribut?**

J Karena pasien rawat ulang adalah jenis khusus dari pasien. Jadi cukup diwakili oleh class PasienRawatUlang, tanpa atribut tambahan.

BAGIAN 3 | FAHMI

Konsep Class & Object (class Orang)

Durasi: ± 3,5 menit • **Konsep:** Class & Object • **Fokus:** orang.py, main.py

Poin yang Dijelaskan

- **Class** adalah cetakan (*blueprint*) yang berisi atribut dan method. **Object** adalah wujud nyata yang dibuat dari class.
- Project ini punya **4 class**: `Orang`, `Pasien`, `PasienRawatUlang`, dan `RekamMedis`, masing-masing di file sendiri.
- `Orang` (file `orang.py`) adalah class induk yang menyimpan data dasar seseorang: `umur` dan `jenis_kelamin`.
- Constructor `__init__` berjalan otomatis saat object dibuat, sedangkan `self` menunjuk ke object itu sendiri.
- Method `tampilkan_info()` menampilkan umur dan jenis kelamin. Method ini nanti di-override oleh class turunan.
- Di `main.py`, setiap baris CSV diubah menjadi object, misalnya `Pasien(patient_id, umur, jenis_kelamin, kepuasan)`. Totalnya **1968 object**: 984 object pasien (720 `Pasien` + 264 `PasienRawatUlang`) dan 984 object `RekamMedis`, semuanya disimpan di list `daftar_rekam`.

Kode & Tampilan yang Ditunjukkan

orang.py — baris 6–13

```
 6  class Orang:
 7      """Menyimpan data dasar seseorang: umur dan jenis kelamin."""
 8
 9      def __init__(self, umur, jenis_kelamin):
10          # [ENKAPSULASI] Nilai diisi lewat setter, jadi langsung divalidasi
11          # sebelum disimpan ke atribut private (self.__umur, self.__jenis_kelamin).
12          self.umur = umur
13          self.jenis_kelamin = jenis_kelamin
```

Fungsi tiap baris · orang.py baris 6–13

Baris	Kode	Fungsinya
6	`class Orang:`	Membuat class baru bernama `Orang`, yaitu cetakan data dasar seseorang.
7	`"""Menyimpan data dasar seseorang: umur dan jenis kelamin."""`	Docstring: keterangan singkat isi class (tidak dijalankan sebagai perintah).
9	`def __init__(self, umur, jenis_kelamin):`	Constructor `__init__`: berjalan otomatis saat object dibuat dan menerima umur serta jenis kelamin. `self` adalah object itu sendiri.
10	`# [ENKAPSULASI] Nilai diisi lewat setter, jadi langsung divalidasi`	Komentar (tidak dijalankan): nilai diisi lewat setter supaya langsung divalidasi.
11	`# sebelum disimpan ke atribut private (self.__umur, self.__jenis_kelamin).`	Komentar lanjutan: nilai yang lolos disimpan di atribut private.
12	`self.umur = umur`	Mengisi umur lewat setter `umur`, jadi nilainya diperiksa dulu sebelum disimpan.
13	`self.jenis_kelamin = jenis_kelamin`	Mengisi jenis kelamin lewat setter `jenis_kelamin`.

orang.py — baris 38–42

```
38      # ---------- Method ----------
39      def tampilkan_info(self):
40          """Menampilkan info dasar. Method ini di-override oleh class turunan."""
41          print(f"Umur           : {self.umur} tahun")
42          print(f"Jenis Kelamin  : {self.jenis_kelamin}")
```

Fungsi tiap baris · orang.py baris 38–42

Baris	Kode	Fungsinya
38	`# ---------- Method ----------`	Komentar penanda bagian method.
39	`def tampilkan_info(self):`	Method `tampilkan_info()` untuk menampilkan data dasar; nanti di-override class turunan.
40	`"""Menampilkan info dasar. Method ini di-override oleh class turunan."""`	Docstring: keterangan method.
41	`print(f"Umur : {self.umur} tahun")`	Mencetak umur (dibaca lewat property `self.umur`).
42	`print(f"Jenis Kelamin : {self.jenis_kelamin}")`	Mencetak jenis kelamin.

main.py — baris 44–48

```
44      # [INHERITANCE] Kolom Readmission menentukan class object yang dibuat
45      if readmission == "Yes":
46          return PasienRawatUlang(patient_id, umur, jenis_kelamin, kepuasan)
47      elif readmission == "No":
48          return Pasien(patient_id, umur, jenis_kelamin, kepuasan)
```

Fungsi tiap baris · main.py baris 44–48

Baris	Kode	Fungsinya
44	`# [INHERITANCE] Kolom Readmission menentukan class object yang dibuat`	Komentar: kolom Readmission menentukan class object yang dibuat.
45	`if readmission == "Yes":`	Jika Readmission bernilai "Yes" …
46	`return PasienRawatUlang(patient_id, umur, jenis_kelamin, kepuasan)`	… buat dan kembalikan object `PasienRawatUlang`.
47	`elif readmission == "No":`	Jika bernilai "No" …
48	`return Pasien(patient_id, umur, jenis_kelamin, kepuasan)`	… buat dan kembalikan object `Pasien`.

Naskah Penyampaian

Sama dengan **Naskah Presentasi Kelompok 3.pdf** dan catatan pembicara PPT. Versi lengkap dengan tanda KLIK dan tunjuk ada di dokumen naskah.

> **Slide 9** *Terima kasih, Yus. Saya Fahmi, akan menjelaskan konsep class dan object.*
>
> **Slide 10** *Class bisa diibaratkan cetakan atau blueprint, sedangkan object adalah hasil cetakannya. Analoginya seperti formulir pendaftaran pasien: bentuk formulirnya sama untuk semua orang, tetapi isinya berbeda untuk setiap pasien. Ini class Pasien sebagai cetakan. Dari satu cetakan, kita bisa membuat banyak object. Baris pertama data menjadi object pasien ID 1, umur 45 tahun, perempuan, dengan kepuasan 4. Baris ketiga menjadi pasien ID 3, umur 32 tahun. Baris keenam menjadi pasien ID 6, laki-laki berumur 68 tahun. Dari seluruh data, terbentuk 720 object Pasien. Di project kami ada empat class: Orang, Pasien, PasienRawatUlang, dan RekamMedis.*

Slide 11 *Ini kode class Orang di file orang.py. Baris 6, class Orang, fungsinya membuat class baru bernama Orang. Baris 7 adalah docstring, yaitu keterangan singkat isi class. Baris 9, def __init__, adalah constructor: method yang berjalan otomatis setiap kali object dibuat, dan menerima umur serta jenis kelamin. Kata self menunjuk ke object itu sendiri, sehingga setiap object menyimpan datanya masing-masing. Baris 10 dan 11 hanya komentar, tidak dijalankan. Baris 12 dan 13 mengisi umur dan jenis kelamin ke object. Nilainya dikirim lewat setter, jadi langsung divalidasi; bagian ini nanti dijelaskan Diva. Lalu baris 39 membuat method tampilkan_info, dan baris 41 serta 42 mencetak umur dan jenis kelamin ke layar. Method ini nanti ditulis ulang, atau di-override, oleh class turunannya.*

Slide 12 *Lalu bagaimana object dibuat dari file CSV? Ini potongan fungsi buat_pasien di main.py. Baris 45 memeriksa kolom Readmission. Kalau nilainya Yes, baris 46 membuat dan mengembalikan object PasienRawatUlang. Kalau nilainya No, baris 48 membuat object Pasien. Jadi pemilihan class terjadi otomatis, dan bagian lain program tidak perlu memeriksa jenis pasien secara manual. Potongan kedua dari fungsi muat_data: baris 73 menyiapkan pembaca CSV, baris 74 mengulang setiap baris data, dan baris 76 membuat object RekamMedis lalu menambahkannya ke list daftar_rekam dengan append. Karena ada 984 baris dan setiap baris menghasilkan dua object, totalnya 1968 object: 720 Pasien, 264 PasienRawatUlang, dan 984 RekamMedis. Selanjutnya, Diva akan menjelaskan bagaimana data di dalam object ini dilindungi dengan enkapsulasi.*

Persiapan Tanya Jawab

T **Apa perbedaan class dan object?**

J Class adalah rancangan (cetakan), sedangkan object adalah hasil nyatanya. Satu class bisa menghasilkan banyak object, contohnya class Pasien yang menghasilkan ratusan object pasien.

T **Apa fungsi self?**

J self menunjuk ke object yang sedang dipakai, sehingga setiap object menyimpan datanya masing-masing.

T **Berapa object yang dibuat program?**

J 1968 object: 984 object pasien (720 Pasien dan 264 PasienRawatUlang) ditambah 984 object RekamMedis.

BAGIAN 4 | DIVA NURISLAMIA AGFA

Enkapsulasi: Atribut Private, Getter & Setter

Durasi: ± 3,5 menit • **Konsep:** Enkapsulasi • **Fokus:** orang.py, pasien.py, main.py

Poin yang Dijelaskan

- **Enkapsulasi** artinya data dibungkus di dalam class dan tidak bisa diubah sembarangan dari luar.
- Semua atribut dibuat **private** dengan awalan dua garis bawah, misalnya `self.__umur` dan `self.__kepuasan`. Python mengganti namanya di dalam menjadi `_Orang__umur` (*name mangling*), jadi tidak bisa diakses langsung.
- Data dibaca lewat **getter** (`@property`) dan diubah lewat **setter** (`@nama.setter`).
- Setiap setter **memvalidasi** data dan memunculkan `ValueError` jika aturan dilanggar: umur harus positif, jenis kelamin Male/Female, kepuasan 1–5, biaya positif, lama rawat bilangan bulat > 0, kondisi dan prosedur tidak boleh kosong, serta hasil Recovered/Stable.
- Constructor juga memakai setter (contohnya `self.umur = umur`), sehingga data dari CSV langsung divalidasi sejak object dibuat.
- Demo di `main.py`: perintah `pasien.kepuasan = 7` ditolak dengan `ValueError`, dan nilai kepuasan **tetap 4**.

Kode & Tampilan yang Ditunjukkan

orang.py — baris 15–25

```
15      # ---------- [ENKAPSULASI] getter & setter ----------
16      @property
17      def umur(self):
18          return self.__umur
19
20      @umur.setter
21      def umur(self, nilai):
22          # Aturan data: Age harus bilangan positif
23          if not isinstance(nilai, int) or nilai <= 0:
24              raise ValueError(f"Umur harus bilangan bulat positif, bukan {nilai}")
25          self.__umur = nilai
```

Fungsi tiap baris · orang.py baris 15–25

Baris	Kode	Fungsinya
15	`# ---------- [ENKAPSULASI] getter & setter ----------`	Komentar penanda bagian getter & setter (enkapsulasi).
16	`@property`	`@property` menjadikan method di bawahnya sebagai getter, sehingga bisa dibaca seperti atribut: `orang.umur`.
17	`def umur(self):`	Getter `umur`: dipanggil saat kode membaca `orang.umur`.
18	`return self.__umur`	Mengembalikan nilai dari atribut private `__umur`.
20	`@umur.setter`	`@umur.setter` menjadikan method di bawahnya sebagai setter untuk umur.
21	`def umur(self, nilai):`	Setter `umur`: dipanggil saat kode menulis `orang.umur = nilai`.
22	`# Aturan data: Age harus bilangan positif`	Komentar: aturan data, umur harus bilangan positif.
23	`if not isinstance(nilai, int) or nilai <= 0:`	Validasi: menolak nilai yang bukan bilangan bulat (`int`) atau yang kurang dari/sama dengan 0.
24	`raise ValueError(f"Umur harus bilangan bulat positif, bukan {nilai}")`	Memunculkan `ValueError` beserta pesan, sehingga nilai salah tidak pernah disimpan.
25	`self.__umur = nilai`	Jika lolos validasi, nilai disimpan ke atribut private `__umur`.

pasien.py — baris 34–39

```
34      @kepuasan.setter
35      def kepuasan(self, nilai):
36          # Aturan data: Satisfaction bernilai 1 sampai 5
37          if not isinstance(nilai, int) or nilai < 1 or nilai > 5:
38              raise ValueError(f"Kepuasan harus bilangan bulat 1 sampai 5, bukan {nilai}")
39          self.__kepuasan = nilai
```

Fungsi tiap baris · pasien.py baris 34–39

Baris	Kode	Fungsinya
34	`@kepuasan.setter`	`@kepuasan.setter` menandai setter untuk kepuasan.
35	`def kepuasan(self, nilai):`	Setter `kepuasan`: dipanggil saat kode menulis `pasien.kepuasan = nilai`.
36	`# Aturan data: Satisfaction bernilai 1 sampai 5`	Komentar: aturan data kepuasan 1 sampai 5.
37	`if not isinstance(nilai, int) or nilai < 1 or nilai > 5:`	Validasi: menolak nilai bukan bilangan bulat atau di luar rentang 1–5.
38	`raise ValueError(f"Kepuasan harus bilangan bulat 1 sampai 5, bukan {nilai}")`	Memunculkan `ValueError` dengan pesan kesalahan.
39	`self.__kepuasan = nilai`	Jika lolos, nilai disimpan ke atribut private `__kepuasan`.

main.py — baris 139–143

```
139     try:
140         pasien.kepuasan = 7  # tidak valid, karena aturan kepuasan 1 sampai 5
141     except ValueError as error:
142         print(f"ValueError tertangkap: {error}")
143     print(f"Kepuasan pasien ID {pasien.patient_id} sesudahnya     : {pasien.kepuasan} (tidak
    berubah)")
```

Fungsi tiap baris · main.py baris 139–143

Baris	Kode	Fungsinya
139	`try:`	`try`: menandai kode yang mungkin menimbulkan error.
140	`pasien.kepuasan = 7  # tidak valid, karena aturan kepuasan 1 sampai 5`	Sengaja mengisi kepuasan dengan 7 (tidak valid, aturannya 1–5); setter akan menolak.
141	`except ValueError as error:`	`except ValueError`: menangkap error dari setter supaya program tidak berhenti.
142	`print(f"ValueError tertangkap: {error}")`	Mencetak pesan error yang tertangkap.
143	`print(f"Kepuasan pasien ID {pasien.patient_id} sesudahnya     : {pasien.kepuasan} (tidak berubah)")`	Mencetak kepuasan setelah percobaan, terbukti tetap 4 (tidak berubah).

Output demo enkapsulasi

```
============================================================
DEMO ENKAPSULASI (setter dengan data tidak valid)
============================================================
Kepuasan pasien ID 1 sebelum diubah : 4
Mencoba mengubah: pasien.kepuasan = 7
ValueError tertangkap: Kepuasan harus bilangan bulat 1 sampai 5, bukan 7
Kepuasan pasien ID 1 sesudahnya     : 4 (tidak berubah)
```

Naskah Penyampaian

Sama dengan **Naskah Presentasi Kelompok 3.pdf** dan catatan pembicara PPT. Versi lengkap dengan tanda KLIK dan tunjuk ada di dokumen naskah.

> **Slide 13** *Terima kasih, Fahmi. Saya Diva, akan menjelaskan enkapsulasi.*

Slide 14 *Enkapsulasi artinya data dibungkus di dalam class dan tidak bisa diubah sembarangan dari luar, supaya data penting seperti umur atau kepuasan selalu diperiksa dulu. Di project kami, semua atribut dibuat private dengan awalan dua garis bawah, misalnya __umur. Ini percobaan langsung di Python dengan class kami. Perintah pertama membuat object pasien ID 1. Perintah kedua membaca kepuasan lewat property, hasilnya 4; inilah cara yang benar. Perintah ketiga mencoba membaca atribut private secara langsung, hasilnya AttributeError, dan Python menyarankan memakai kepuasan. Perintah terakhir menampilkan nama semua atribut di dalam object. Namanya sudah diubah oleh Python, misalnya __umur menjadi _Orang__umur. Ini disebut name mangling.*

Slide 15 *Lalu bagaimana membaca dan mengubah atribut private? Ini kodenya untuk umur. Baris 16, @property, menjadikan method di bawahnya sebagai getter. Baris 17 dan 18 berfungsi membaca umur, yaitu mengembalikan nilai dari __umur. Baris 20, @umur.setter, menjadikan method di bawahnya sebagai setter, yang dipanggil setiap kali kita menulis pasien titik umur sama dengan suatu nilai. Baris 23 adalah validasi: memeriksa apakah nilainya bilangan bulat dan lebih dari nol. Kalau tidak, baris 24 memunculkan ValueError, sehingga nilai yang salah tidak tersimpan. Kalau lolos, baris 25 menyimpan nilainya ke __umur. Pola yang sama kami pakai di semua setter, misalnya kepuasan satu sampai lima dan lama rawat minimal satu hari. Constructor juga memakai setter, jadi data dari CSV langsung divalidasi sejak object dibuat.*

Slide 16 *Sekarang buktinya, dari fungsi demo di main.py. Baris 139, try, menandai kode yang mungkin menimbulkan error. Baris 140 sengaja mengisi kepuasan pasien ID 1 dengan 7, padahal aturannya satu sampai lima. Baris 141, except ValueError, menangkap error dari setter supaya program tidak berhenti, dan baris 142 mencetak pesan errornya. Di output terlihat pesan ValueError, dan di baris terakhir nilai kepuasan tetap 4. Tanpa enkapsulasi, nilai tujuh itu akan langsung tersimpan dan merusak perhitungan, misalnya rata-rata kepuasan. Jadi manfaat utama enkapsulasi adalah data selalu dalam keadaan valid. Selanjutnya, Rosa akan menjelaskan inheritance.*

Persiapan Tanya Jawab

T **Apa yang terjadi jika kita menulis print(pasien.__kepuasan)?**

J Muncul AttributeError karena nama atribut private sudah diganti oleh Python (name mangling). Cara yang benar adalah pasien.kepuasan, yaitu lewat property.

T **Kenapa constructor memanggil setter, bukan langsung mengisi self.__umur?**

J Supaya validasi juga berlaku saat object pertama kali dibuat. Jadi data CSV yang salah langsung tertolak.

T **Kenapa memakai ValueError?**

J Karena yang salah adalah nilai datanya. ValueError adalah exception bawaan Python untuk nilai yang tidak sesuai, dan bisa ditangkap dengan try-except.

BAGIAN 5 | ROSA RAMADANI

Inheritance (1): Pasien Mewarisi Orang

Durasi: ± 3 menit • **Konsep:** Inheritance • **Fokus:** pasien.py

Poin yang Dijelaskan

- **Inheritance** (pewarisan) artinya class anak mewarisi atribut dan method dari class induk, sehingga kode tidak perlu ditulis ulang.
- `class Pasien(Orang)`: Pasien otomatis memiliki `umur` dan `jenis_kelamin` beserta getter, setter, dan validasinya dari Orang.
- Pasien menambah atribut sendiri: `patient_id` dan `kepuasan`.
- **Constructor chaining**: `super().__init__(umur, jenis_kelamin)` menjalankan constructor Orang untuk mengisi data dasar, lalu Pasien mengisi `patient_id` dan `kepuasan`.
- **Method overriding**: Pasien menulis ulang `tampilkan_info()`. Di dalamnya, `super().tampilkan_info()` dipanggil agar tampilan umur dan jenis kelamin dari Orang tetap dipakai, lalu ditambah ID pasien dan kepuasan.

Kode & Tampilan yang Ditunjukkan

pasien.py — baris 9–17

```
 9  class Pasien(Orang):
10      """Pasien rumah sakit (turunan dari Orang)."""
11
12      def __init__(self, patient_id, umur, jenis_kelamin, kepuasan):
13          # [INHERITANCE] Constructor chaining: umur & jenis_kelamin diurus oleh Orang
14          super().__init__(umur, jenis_kelamin)
15          # [ENKAPSULASI] Diisi lewat setter agar tervalidasi
16          self.patient_id = patient_id
17          self.kepuasan = kepuasan
```

Fungsi tiap baris · pasien.py baris 9–17

Baris	Kode	Fungsinya
9	`class Pasien(Orang):`	Membuat class `Pasien` yang mewarisi `Orang` (nama induk ditulis di dalam kurung).
10	`"""Pasien rumah sakit (turunan dari Orang)."""`	Docstring: keterangan class.
12	`def __init__(self, patient_id, umur, jenis_kelamin, kepuasan):`	Constructor `Pasien`: menerima empat data, yaitu ID, umur, jenis kelamin, dan kepuasan.
13	`# [INHERITANCE] Constructor chaining: umur & jenis_kelamin diurus oleh Orang`	Komentar: umur & jenis kelamin diurus oleh class Orang.
14	`super().__init__(umur, jenis_kelamin)`	`super().__init__(...)` menjalankan constructor milik `Orang` (constructor chaining) untuk mengisi umur dan jenis kelamin.
15	`# [ENKAPSULASI] Diisi lewat setter agar tervalidasi`	Komentar: atribut tambahan diisi lewat setter.
16	`self.patient_id = patient_id`	Mengisi `patient_id` lewat setter-nya (harus bilangan bulat positif).
17	`self.kepuasan = kepuasan`	Mengisi `kepuasan` lewat setter-nya (harus 1 sampai 5).

pasien.py — baris 41–46

```
41      # ---------- [INHERITANCE] Method overriding ----------
42      def tampilkan_info(self):
43          """Override tampilkan_info() milik Orang: tambah ID pasien dan kepuasan."""
44          print(f"ID Pasien      : {self.patient_id}")
45          super().tampilkan_info()  # memakai ulang tampilan umur & jenis kelamin dari Orang
46          print(f"Kepuasan       : {self.kepuasan}/5")
```

Fungsi tiap baris · pasien.py baris 41–46

Baris	Kode	Fungsinya
41	`# ---------- [INHERITANCE] Method overriding ----------`	Komentar penanda method overriding.
42	`def tampilkan_info(self):`	Menulis ulang (override) method `tampilkan_info()` milik `Orang`.
43	`"""Override tampilkan_info() milik Orang: tambah ID pasien dan kepuasan."""`	Docstring: keterangan method override.
44	`print(f"ID Pasien : {self.patient_id}")`	Mencetak ID pasien.
45	`super().tampilkan_info()  # memakai ulang tampilan umur & jenis kelamin dari Orang`	Memanggil `tampilkan_info()` milik `Orang` lewat `super()`, sehingga umur dan jenis kelamin ikut tercetak.
46	`print(f"Kepuasan : {self.kepuasan}/5")`	Mencetak kepuasan dalam format skala 5, misalnya `4/5`.

Hasil tampilkan_info() pada object Pasien (rekam medis ke-1)

```
[1] Object pasien dari class: Pasien
--- Data Pasien ---
ID Pasien      : 1
Umur           : 45 tahun
Jenis Kelamin  : Female
Kepuasan       : 4/5
--- Data Medis ---
Kondisi        : Heart Disease
Prosedur       : Angioplasty
Biaya          : 15,000.00
Lama Rawat     : 5 hari
Biaya per Hari : 3,000.00
Hasil          : Recovered
```

Naskah Penyampaian

Sama dengan **Naskah Presentasi Kelompok 3.pdf** dan catatan pembicara PPT. Versi lengkap dengan tanda KLIK dan tunjuk ada di dokumen naskah.

Slide 17 *Terima kasih, Diva. Saya Rosa, akan menjelaskan inheritance bagian pertama.*

Slide 18 *Inheritance atau pewarisan artinya class anak mendapatkan atribut dan method dari class induknya, seperti anak yang mewarisi sifat orang tuanya lalu punya ciri khasnya sendiri. Di project kami, class Pasien mewarisi class Orang; panah bersegitiga ini berarti mewarisi. Jadi Pasien otomatis punya umur dan jenis kelamin beserta validasinya, dan hanya menambah patient_id serta kepuasan. Keuntungannya, kalau aturan validasi umur diubah di Orang, perubahan itu otomatis berlaku untuk semua pasien. Sekarang kodenya. Baris 9, class Pasien(Orang), fungsinya membuat class Pasien yang mewarisi Orang; nama induknya ditulis di dalam kurung. Baris 12 adalah constructor Pasien yang menerima empat data. Baris 14, super().__init__, menjalankan constructor milik Orang untuk mengisi umur dan jenis kelamin. Ini disebut constructor chaining. Kalau baris ini lupa ditulis, atribut umur dan jenis kelamin tidak pernah dibuat dan program akan error. Baris 16 dan 17 mengisi patient_id dan kepuasan lewat setter milik Pasien.*

Slide 19 *Selain mewarisi, Pasien juga melakukan method overriding, yaitu menulis ulang method milik induknya dengan nama yang sama. Baris 42 membuat ulang method tampilkan_info di class Pasien. Baris 44 mencetak ID pasien. Baris 45 memanggil versi milik Orang lewat super().tampilkan_info(), sehingga umur dan jenis kelamin ikut tercetak tanpa menyalin kode. Baris 46 mencetak kepuasan dengan format per lima. Hasilnya terlihat di sini pada pasien pertama dari data asli: baris umur dan jenis kelamin berasal dari Orang, sedangkan ID pasien dan kepuasan ditambahkan oleh Pasien. Jadi overriding membuat kita bisa memakai ulang kode induk sekaligus menambah perilaku baru. Perlu dibedakan, overriding tidak sama dengan overloading: overriding menulis ulang method induk di class anak dengan nama yang sama. Selanjutnya, Sofiang akan menjelaskan pewarisan bertingkat dan polimorfisme.*

Persiapan Tanya Jawab

T **Apa yang terjadi jika super().__init__() tidak dipanggil?**

J Constructor Orang tidak berjalan, jadi atribut umur dan jenis_kelamin tidak pernah dibuat. Saat diakses akan muncul AttributeError.

T **Apa itu method overriding?**

J Class anak menulis ulang method dengan nama yang sama milik class induk untuk mengubah atau menambah perilakunya, seperti tampilkan_info() di Pasien.

T **Apa bedanya overriding dan overloading?**

J Overriding = menulis ulang method induk di class anak. Overloading = beberapa method bernama sama dengan parameter berbeda dalam satu class, yang tidak didukung langsung oleh Python seperti di Java.

BAGIAN 6 | SOFIANG

Inheritance (2): Pewarisan Bertingkat & Polimorfisme

Durasi: ± 2,5 menit • **Konsep:** Inheritance • **Fokus:** pasien_rawat_ulang.py, main.py

Poin yang Dijelaskan

- **Pewarisan bertingkat** (*multilevel*): `Orang` → `Pasien` → `PasienRawatUlang`.
- `PasienRawatUlang` dipakai untuk **264 baris** data dengan Readmission = Yes. Di `main.py`, nilai kolom Readmission menentukan class object yang dibuat.
- Class ini **tidak menambah atribut** karena CSV tidak punya kolom tambahan, tetapi punya **perilaku khusus**: method `status_rawat()` yang mengembalikan teks "Pasien rawat ulang". Method ini tidak dimiliki `Pasien` maupun `Orang`.
- `super().__init__()` meneruskan data ke Pasien, lalu Pasien meneruskannya ke Orang, sehingga terbentuk rantai constructor 3 tingkat.
- Override `tampilkan_info()`: memanggil versi milik Pasien, lalu menambah baris **"Status Rawat : Pasien rawat ulang"** yang isinya diambil dari `status_rawat()`.
- **Polimorfisme**: `RekamMedis` cukup memanggil `self.pasien.tampilkan_info()`, dan Python otomatis menjalankan versi yang sesuai dengan class object-nya. Hasilnya, baris "Status Rawat" hanya muncul pada object `PasienRawatUlang`.
- Urutan pencarian method (MRO): `PasienRawatUlang` → `Pasien` → `Orang` → `object`.

Kode & Tampilan yang Ditunjukkan

pasien_rawat_ulang.py — baris 10–26

```
10  class PasienRawatUlang(Pasien):
11      """Pasien yang kembali dirawat (Readmission = Yes)."""
12
13      def __init__(self, patient_id, umur, jenis_kelamin, kepuasan):
14          # [INHERITANCE] Constructor chaining: Pasien -> Orang
15          super().__init__(patient_id, umur, jenis_kelamin, kepuasan)
16
17      # ---------- Method khusus PasienRawatUlang ----------
18      def status_rawat(self):
19          """Mengembalikan keterangan status rawat pasien ini."""
20          return "Pasien rawat ulang"
21
22      # ---------- [INHERITANCE] Method overriding ----------
23      def tampilkan_info(self):
24          """Override tampilkan_info() milik Pasien: tambah status_rawat()."""
25          super().tampilkan_info()  # tampilkan ID, umur, jenis kelamin, kepuasan
26          print(f"Status Rawat   : {self.status_rawat()}")
```

Fungsi tiap baris · pasien_rawat_ulang.py baris 10–26

Baris	Kode	Fungsinya
10	`class PasienRawatUlang(Pasien):`	Membuat class `PasienRawatUlang` yang mewarisi `Pasien` (pewarisan bertingkat: Orang → Pasien → PasienRawatUlang).
11	`"""Pasien yang kembali dirawat (Readmission = Yes)."""`	Docstring: class ini dipakai untuk baris data dengan Readmission = Yes.
13	`def __init__(self, patient_id, umur, jenis_kelamin, kepuasan):`	Constructor dengan empat data yang sama seperti `Pasien`.
14	`# [INHERITANCE] Constructor chaining: Pasien -> Orang`	Komentar: rantai constructor dari Pasien ke Orang.
15	`super().__init__(patient_id, umur, jenis_kelamin, kepuasan)`	Meneruskan semua data ke constructor `Pasien`, yang lalu meneruskan umur & jenis kelamin ke `Orang`.

Baris	Kode	Fungsinya
17	`# ---------- Method khusus PasienRawatUlang ----------`	Komentar penanda method khusus, yaitu method yang hanya dimiliki `PasienRawatUlang`.
18	`def status_rawat(self):`	Method `status_rawat()`: perilaku khas `PasienRawatUlang`, tidak ada di `Pasien` maupun `Orang`.
19	`"""Mengembalikan keterangan status rawat pasien ini."""`	Docstring: keterangan method.
20	`return "Pasien rawat ulang"`	Mengembalikan teks “Pasien rawat ulang” sebagai keterangan status pasien ini.
22	`# ---------- [INHERITANCE] Method overriding ----------`	Komentar penanda method overriding.
23	`def tampilkan_info(self):`	Override `tampilkan_info()` milik `Pasien`.
24	`"""Override tampilkan_info() milik Pasien: tambah status_rawat()."""`	Docstring: keterangan method override.
25	`super().tampilkan_info()  # tampilkan ID, umur, jenis kelamin, kepuasan`	Memanggil versi `Pasien` lewat `super()` untuk mencetak ID, umur, jenis kelamin, dan kepuasan.
26	`print(f"Status Rawat  : {self.status_rawat()}")`	Menambahkan baris `Status Rawat`, isinya diambil dari method `status_rawat()`.

main.py — baris 44–50

```python
    # [INHERITANCE] Kolom Readmission menentukan class object yang dibuat
    if readmission == "Yes":
        return PasienRawatUlang(patient_id, umur, jenis_kelamin, kepuasan)
    elif readmission == "No":
        return Pasien(patient_id, umur, jenis_kelamin, kepuasan)
    else:
        raise ValueError(f"Readmission harus 'Yes' atau 'No', bukan '{readmission}'")
```

Fungsi tiap baris · main.py baris 44–50

Baris	Kode	Fungsinya
44	`# [INHERITANCE] Kolom Readmission menentukan class object yang dibuat`	Komentar: kolom Readmission menentukan class object yang dibuat.
45	`if readmission == "Yes":`	Jika Readmission bernilai "Yes" …
46	`return PasienRawatUlang(patient_id, umur, jenis_kelamin, kepuasan)`	… buat dan kembalikan object `PasienRawatUlang`.
47	`elif readmission == "No":`	Jika bernilai "No" …
48	`return Pasien(patient_id, umur, jenis_kelamin, kepuasan)`	… buat dan kembalikan object `Pasien`.
49	`else:`	Selain Yes/No …
50	`raise ValueError(f"Readmission harus 'Yes' atau 'No', bukan '{readmission}'")`	… nilainya tidak valid, jadi memunculkan `ValueError`.

Hasil tampilkan_info() pada object PasienRawatUlang (rekam medis ke-2)

```
[2] Object pasien dari class: PasienRawatUlang
--- Data Pasien ---
ID Pasien      : 2
Umur           : 60 tahun
Jenis Kelamin  : Male
Kepuasan       : 3/5
Status Rawat   : Pasien rawat ulang
--- Data Medis ---
Kondisi        : Diabetes
Prosedur       : Insulin Therapy
Biaya          : 2,000.00
Lama Rawat     : 3 hari
Biaya per Hari : 666.67
Hasil          : Stable
```

Naskah Penyampaian

Sama dengan **Naskah Presentasi Kelompok 3.pdf** dan catatan pembicara PPT. Versi lengkap dengan tanda KLIK dan tunjuk ada di dokumen naskah.

Slide 20 *Terima kasih, Rosa. Saya Sofiang, melanjutkan inheritance bagian kedua.*

Slide 21 *Inheritance di project kami bertingkat. Orang adalah induk dari Pasien, dan Pasien adalah induk dari PasienRawatUlang. Jadi PasienRawatUlang mewarisi semua milik Pasien dan Orang sekaligus, dan satu object PasienRawatUlang juga dianggap sebagai Pasien dan sebagai Orang. Class ini dipakai untuk 264 pasien yang Readmission-nya Yes. Sekarang kodenya. Baris 10, class PasienRawatUlang(Pasien), membuat class yang mewarisi Pasien. Baris 13 adalah constructor-nya, dan baris 15 memanggil super().__init__ untuk meneruskan semua data ke Pasien, lalu Pasien meneruskan umur dan jenis kelamin ke Orang. Class ini tidak menambah atribut, karena data CSV tidak punya kolom tambahan, tetapi punya perilaku khusus. Baris 18 membuat method status_rawat, dan baris 20 mengembalikan teks Pasien rawat ulang. Method ini tidak dimiliki Pasien maupun Orang. Baris 23 meng-override tampilkan_info. Baris 25 memanggil versi milik Pasien lewat super, dan baris 26 menambahkan baris Status Rawat yang isinya diambil dari status_rawat.*

Slide 22 *Hasil overriding itu terlihat di sini. Rekam pertama adalah object Pasien, tampilannya berhenti di kepuasan. Rekam kedua adalah object PasienRawatUlang, dan ada tambahan baris Status Rawat: Pasien rawat ulang. Padahal di class RekamMedis, baris 98 hanya berisi satu perintah, self.pasien.tampilkan_info(), yang fungsinya memanggil tampilkan_info milik object pasien yang tersimpan di rekam medis. Baris 96 dan 97 di atasnya hanya komentar. Python otomatis memilih versi method sesuai class object-nya. Inilah polimorfisme: satu perintah, hasilnya berbeda. Tanpa polimorfisme, kita harus menulis pengecekan if untuk setiap jenis pasien. Urutan Python mencari method disebut MRO: dimulai dari PasienRawatUlang, lalu Pasien, lalu Orang, sehingga versi PasienRawatUlang yang dipakai lebih dulu. Selanjutnya, Revalinda akan menjelaskan class RekamMedis.*

Persiapan Tanya Jawab

T **Kenapa PasienRawatUlang tidak punya atribut baru?**

J Karena aturan project melarang atribut di luar kolom data. Yang membedakannya adalah perilaku: method khusus status_rawat() dan override tampilkan_info() yang menampilkan statusnya.

T **Kenapa pasien biasa tidak menampilkan baris Status Rawat?**

J Karena baris itu hanya ditambahkan oleh override tampilkan_info() di class PasienRawatUlang, dan method status_rawat() hanya ada di class itu. Inilah efek overriding.

T **Apa itu MRO?**

J Method Resolution Order, yaitu urutan Python mencari method: PasienRawatUlang → Pasien → Orang → object.

BAGIAN 7 | REVALINDA

Komposisi & Validasi: class RekamMedis

Durasi: ± 3,5 menit • **Konsep:** Class & Object (komposisi) • **Fokus:** rekam_medis.py

Poin yang Dijelaskan

- `RekamMedis` **tidak mewarisi** class apa pun, tetapi memakai **komposisi**: di dalamnya tersimpan object `Pasien`. Hubungannya *has-a* (rekam medis **memiliki** pasien), berbeda dengan inheritance yang *is-a* (pasien **adalah** orang).
- Atribut (semuanya private + property/setter): `pasien`, `kondisi`, `prosedur`, `biaya`, `lama_rawat`, dan `hasil`.
- Setter `pasien` memakai `isinstance(nilai, Pasien)`, sehingga object `PasienRawatUlang` juga diterima karena termasuk turunan Pasien.
- **Validasi data medis** di setter: kondisi dan prosedur tidak boleh kosong (teks berisi spasi saja juga ditolak karena `strip()`), biaya harus positif, lama rawat bilangan bulat lebih dari 0, dan hasil hanya Recovered/Stable. Pelanggaran memunculkan `ValueError`.
- `hitung_biaya_per_hari()` = biaya ÷ lama rawat. Contoh pasien ID 1: 15,000 ÷ 5 hari = 3,000 per hari. Validasi lama rawat mencegah pembagian dengan nol.
- `tampilkan_rekam()` menampilkan data pasien (lewat `tampilkan_info()` milik object pasien) dan data medis.

Kode & Tampilan yang Ditunjukkan

rekam_medis.py — baris 9–19

```
 9  class RekamMedis:
10      """Rekam medis satu perawatan milik seorang pasien."""
11
12      def __init__(self, pasien, kondisi, prosedur, biaya, lama_rawat, hasil):
13          # [ENKAPSULASI] Semua nilai diisi lewat setter agar tervalidasi
14          self.pasien = pasien
15          self.kondisi = kondisi
16          self.prosedur = prosedur
17          self.biaya = biaya
18          self.lama_rawat = lama_rawat
19          self.hasil = hasil
```

Fungsi tiap baris · rekam_medis.py baris 9–19

Baris	Kode	Fungsinya
9	`class RekamMedis:`	Membuat class `RekamMedis` (tanpa induk; memakai komposisi).
10	`"""Rekam medis satu perawatan milik seorang pasien."""`	Docstring: keterangan class.
12	`def __init__(self, pasien, kondisi, prosedur, biaya, lama_rawat, hasil):`	Constructor: menerima object pasien dan lima data medis.
13	`# [ENKAPSULASI] Semua nilai diisi lewat setter agar tervalidasi`	Komentar: semua nilai diisi lewat setter agar tervalidasi.
14	`self.pasien = pasien`	Menyimpan object `Pasien` di dalam rekam medis (komposisi "memiliki").
15	`self.kondisi = kondisi`	Mengisi kondisi penyakit (tidak boleh kosong).
16	`self.prosedur = prosedur`	Mengisi prosedur (tidak boleh kosong).
17	`self.biaya = biaya`	Mengisi biaya (harus bilangan positif).
18	`self.lama_rawat = lama_rawat`	Mengisi lama rawat (minimal 1 hari).
19	`self.hasil = hasil`	Mengisi hasil perawatan (`Recovered` atau `Stable`).

rekam_medis.py — baris 26–31

```
26      @pasien.setter
27      def pasien(self, nilai):
28          # Menerima object Pasien maupun turunannya (PasienRawatUlang)
29          if not isinstance(nilai, Pasien):
30              raise ValueError("Pasien harus berupa object dari class Pasien")
31          self.__pasien = nilai
```

Fungsi tiap baris · rekam_medis.py baris 26–31

Baris	Kode	Fungsinya
26	`@pasien.setter`	`@pasien.setter` menandai setter untuk atribut pasien.
27	`def pasien(self, nilai):`	Setter `pasien`: dipanggil saat kode menulis `rekam.pasien = nilai`.
28	`# Menerima object Pasien maupun turunannya (PasienRawatUlang)`	Komentar: object Pasien maupun turunannya diterima.
29	`if not isinstance(nilai, Pasien):`	Validasi `isinstance`: memeriksa apakah nilai adalah object `Pasien` (termasuk `PasienRawatUlang`).
30	`raise ValueError("Pasien harus berupa object dari class Pasien")`	Jika bukan object Pasien, memunculkan `ValueError`.
31	`self.__pasien = nilai`	Jika lolos, object disimpan ke atribut private `__pasien`.

rekam_medis.py — baris 37–42

```
37      @kondisi.setter
38      def kondisi(self, nilai):
39          # Aturan data: Condition tidak boleh kosong
40          if not isinstance(nilai, str) or nilai.strip() == "":
41              raise ValueError("Kondisi tidak boleh kosong")
42          self.__kondisi = nilai
```

Fungsi tiap baris · rekam_medis.py baris 37–42

Baris	Kode	Fungsinya
37	`@kondisi.setter`	`@kondisi.setter` menandai setter untuk kondisi.
38	`def kondisi(self, nilai):`	Setter `kondisi`: dipanggil saat kode menulis `rekam.kondisi = nilai`.
39	`# Aturan data: Condition tidak boleh kosong`	Komentar: aturan data, Condition tidak boleh kosong.
40	`if not isinstance(nilai, str) or nilai.strip() == "":`	Validasi: nilai harus teks (`str`) dan tidak kosong; `strip()` membuang spasi di awal dan akhir, jadi isian yang hanya berisi spasi juga dianggap kosong.
41	`raise ValueError("Kondisi tidak boleh kosong")`	Jika kosong, memunculkan `ValueError` dengan pesan “Kondisi tidak boleh kosong”.
42	`self.__kondisi = nilai`	Jika lolos, nilai disimpan ke atribut private `__kondisi`.

rekam_medis.py — baris 70–75

```
70      @lama_rawat.setter
71      def lama_rawat(self, nilai):
72          # Aturan data: Length_of_Stay bulat > 0 (mencegah pembagian nol)
73          if not isinstance(nilai, int) or nilai <= 0:
74              raise ValueError(f"Lama rawat harus bilangan bulat positif, bukan {nilai}")
75          self.__lama_rawat = nilai
```

Fungsi tiap baris · rekam_medis.py baris 70–75

Baris	Kode	Fungsinya
70	`@lama_rawat.setter`	`@lama_rawat.setter` menandai setter untuk lama rawat.
71	`def lama_rawat(self, nilai):`	Setter `lama_rawat`: dipanggil saat kode menulis `rekam.lama_rawat = nilai`.
72	`# Aturan data: Length_of_Stay bulat > 0 (mencegah pembagian nol)`	Komentar: aturan data, Length_of_Stay bilangan bulat lebih dari 0, sekaligus mencegah pembagian dengan nol.
73	`if not isinstance(nilai, int) or nilai <= 0:`	Validasi: menolak nilai yang bukan bilangan bulat (`int`) atau yang kurang dari/sama dengan 0.

Baris	Kode	Fungsinya
74	`raise ValueError(f"Lama rawat harus bilangan bulat positif, bukan {nilai}")`	Memunculkan `ValueError` beserta nilai yang salah.
75	`self.__lama_rawat = nilai`	Jika lolos, nilai disimpan ke atribut private `__lama_rawat`.

rekam_medis.py — baris 88–105

```
88      # ---------- Method ----------
89      def hitung_biaya_per_hari(self):
90          """Menghitung biaya rata-rata untuk setiap hari rawat."""
91          return self.biaya / self.lama_rawat
92
93      def tampilkan_rekam(self):
94          """Menampilkan data pasien dan data medis."""
95          print("--- Data Pasien ---")
96          # Polimorfisme: versi tampilkan_info() yang dipanggil
97          # menyesuaikan class object-nya (Pasien atau PasienRawatUlang).
98          self.pasien.tampilkan_info()
99          print("--- Data Medis ---")
100         print(f"Kondisi        : {self.kondisi}")
101         print(f"Prosedur       : {self.prosedur}")
102         print(f"Biaya          : {self.biaya:,.2f}")
103         print(f"Lama Rawat     : {self.lama_rawat} hari")
104         print(f"Biaya per Hari : {self.hitung_biaya_per_hari():,.2f}")
105         print(f"Hasil          : {self.hasil}")
```

Fungsi tiap baris · rekam_medis.py baris 88–105

Baris	Kode	Fungsinya
88	`# ---------- Method ----------`	Komentar penanda bagian method.
89	`def hitung_biaya_per_hari(self):`	Method `hitung_biaya_per_hari()`.
90	`"""Menghitung biaya rata-rata untuk setiap hari rawat."""`	Docstring: keterangan method.
91	`return self.biaya / self.lama_rawat`	Mengembalikan biaya dibagi lama rawat; aman karena lama rawat minimal 1 hari.
93	`def tampilkan_rekam(self):`	Method `tampilkan_rekam()` untuk mencetak satu rekam medis lengkap.
94	`"""Menampilkan data pasien dan data medis."""`	Docstring: keterangan method.
95	`print("--- Data Pasien ---")`	Mencetak judul bagian data pasien.
96	`# Polimorfisme: versi tampilkan_info() yang dipanggil`	Komentar: penjelasan polimorfisme.
97	`# menyesuaikan class object-nya (Pasien atau PasienRawatUlang).`	Komentar lanjutan: versi method mengikuti class object-nya.
98	`self.pasien.tampilkan_info()`	Memanggil `tampilkan_info()` milik object pasien; Python memilih versi `Pasien` atau `PasienRawatUlang` (polimorfisme).
99	`print("--- Data Medis ---")`	Mencetak judul bagian data medis.
100	`print(f"Kondisi : {self.kondisi}")`	Mencetak kondisi.
101	`print(f"Prosedur : {self.prosedur}")`	Mencetak prosedur.
102	`print(f"Biaya : {self.biaya:,.2f}")`	Mencetak biaya dengan pemisah ribuan dan 2 angka desimal.
103	`print(f"Lama Rawat : {self.lama_rawat} hari")`	Mencetak lama rawat dalam hari.
104	`print(f"Biaya per Hari : {self.hitung_biaya_per_hari():,.2f}")`	Mencetak hasil `hitung_biaya_per_hari()` dengan 2 angka desimal.
105	`print(f"Hasil : {self.hasil}")`	Mencetak hasil perawatan.

Naskah Penyampaian

Sama dengan **Naskah Presentasi Kelompok 3.pdf** dan catatan pembicara PPT. Versi lengkap dengan tanda KLIK dan tunjuk ada di dokumen naskah.

> **Slide 23** *Terima kasih, Sofiang. Saya Revalinda, akan menjelaskan komposisi pada class RekamMedis.*

Slide 24 *Berbeda dari tiga class sebelumnya, RekamMedis tidak mewarisi class apa pun. Inheritance adalah hubungan adalah: Pasien adalah Orang. Sedangkan RekamMedis memakai komposisi, yaitu hubungan memiliki: rekam medis memiliki data pasien. Kalimat rekam medis adalah pasien tidak masuk akal, sedangkan rekam medis memiliki pasien masuk akal; ini cara mudah memilih antara keduanya. Di dalam object RekamMedis tersimpan object Pasien, ditambah kondisi, prosedur, biaya, lama rawat, dan hasil. Sekarang kodenya, yaitu setter untuk atribut pasien. Baris 26, @pasien.setter, menandai setter, dan baris 27 menerima nilai baru. Baris 29 memeriksa dengan isinstance: apakah nilai itu object Pasien. Kalau bukan, baris 30 memunculkan ValueError. Kalau benar, baris 31 menyimpannya ke atribut private __pasien. Karena PasienRawatUlang adalah turunan Pasien, object PasienRawatUlang juga diterima.*

Slide 25 *Selain pasien, data medis juga dijaga oleh setter. Ini setter kondisi. Baris 39 adalah komentar aturannya: Condition tidak boleh kosong. Baris 40 memeriksa apakah nilainya teks dan tidak kosong; strip membuang spasi, jadi isian yang hanya berisi spasi juga dianggap kosong. Kalau kosong, baris 41 memunculkan ValueError; kalau lolos, baris 42 menyimpannya. Setter prosedur memakai pola yang sama di baris 48 sampai 53. Untuk lama rawat, baris 73 menolak nilai yang bukan bilangan bulat atau kurang dari satu, lalu baris 74 memunculkan ValueError. Jadi setiap data medis punya aturan, dan baris CSV yang melanggarnya akan dilewati oleh main.py.*

Slide 26 *RekamMedis punya dua method. Baris 89 membuat method hitung_biaya_per_hari, baris 90 adalah docstring keterangannya, dan baris 91 mengembalikan hasil biaya dibagi lama rawat. Perhitungannya memakai property biaya dan lama_rawat, jadi tetap mengikuti enkapsulasi. Contohnya pasien ID 1: biayanya lima belas ribu dan dirawat lima hari, jadi biaya per harinya tiga ribu. Tabel di kanan menunjukkan hasilnya untuk lima pasien pertama. Pembagiannya aman, karena tadi setter lama rawat sudah menolak nilai di bawah satu hari. Method kedua, tampilkan_rekam, menampilkan data pasien lewat tampilkan_info milik object pasien, lalu data medisnya. Selanjutnya, Andi akan menjelaskan program utama di main.py.*

Persiapan Tanya Jawab

T **Kenapa RekamMedis tidak mewarisi Pasien?**

J Karena rekam medis bukan jenis pasien. Hubungannya “memiliki” (has-a), bukan “adalah” (is-a), sehingga yang tepat adalah komposisi.

T **Bagaimana jika lama_rawat bernilai 0?**

J Setter lama_rawat menolaknya dengan ValueError, sehingga hitung_biaya_per_hari() tidak pernah membagi dengan nol.

T **Kenapa kondisi yang hanya berisi spasi juga ditolak?**

J Karena setter memeriksa nilai.strip() == "". strip() membuang spasi di awal dan akhir, jadi teks yang isinya hanya spasi dianggap kosong.

T **Kenapa setter pasien bisa menerima PasienRawatUlang?**

J Karena PasienRawatUlang adalah turunan Pasien, sehingga isinstance(nilai, Pasien) bernilai True.

BAGIAN 8 | ANDI REZKY NABILA

Program Utama (main.py): Membaca CSV & Menu Interaktif

Durasi: ± 3,5 menit • **Konsep:** Class & Object • **Fokus:** main.py

Poin yang Dijelaskan

- `main.py` mengimpor class dari file lain, misalnya `from pasien import Pasien`.
- `cari_file_csv()` mencari file CSV di folder yang sama dengan `main.py`. Dua variasi nama didukung, yaitu memakai garis bawah atau memakai spasi.
- `muat_data()` membuka file dengan modul bawaan `csv`. `csv.DictReader` membaca setiap baris sebagai *dictionary* berdasarkan nama kolom, misalnya `baris["Age"]`. Pengaturan `restval=""` membuat kolom yang hilang dianggap teks kosong, sehingga ikut ditolak validasi dan tidak menghentikan program.
- Isi CSV berupa teks, jadi angka diubah dulu dengan `int()` dan `float()`.
- `buat_pasien()` memilih class sesuai kolom Readmission: Yes → `PasienRawatUlang`, No → `Pasien`, selain itu `ValueError`. `buat_rekam_medis()` lalu memasukkan object pasien ke dalam `RekamMedis`.
- Semua object disimpan di list `daftar_rekam`. Baris yang tidak valid ditangkap dengan `try-except`: dilewati dengan peringatan, **nomor barisnya dicatat** di list `baris_dilewati`, dan **jumlahnya ditampilkan** setelah data dimuat.
- **Menu interaktif** `jalankan_menu()`: `while True` menampilkan menu berulang. 1 = lihat 10 data pertama, 2 = cari pasien berdasarkan Patient_ID, 3 = tambah pasien baru, 4 = keluar (`break`). Pilihan lain dijawab dengan pesan, bukan error.
- Fungsi `main()` mengatur urutan kerja program (lihat flowchart): cari file → muat data → tampilkan jumlah baris dilewati → 5 rekam → ringkasan → demo enkapsulasi → menu.

Kode & Tampilan yang Ditunjukkan

main.py — baris 13–17

```
13  # File CSV dicari di folder yang sama dengan main.py.
14  # Dua variasi nama didukung: memakai garis bawah atau memakai spasi.
15  FOLDER_PROJECT = os.path.dirname(os.path.abspath(__file__))
16  NAMA_FILE_CSV = ["hospital_data_analysis.csv", "hospital data analysis.csv"]
17  JUMLAH_DITAMPILKAN = 5
```

Fungsi tiap baris · main.py baris 13–17

Baris	Kode	Fungsinya
13	`# File CSV dicari di folder yang sama dengan main.py.`	Komentar: file CSV dicari di folder yang sama dengan main.py.
14	`# Dua variasi nama didukung: memakai garis bawah atau memakai spasi.`	Komentar: dua variasi nama file didukung.
15	`FOLDER_PROJECT = os.path.dirname(os.path.abspath(__file__))`	Menyimpan lokasi folder tempat main.py berada, supaya program tetap jalan walau dijalankan dari folder lain.
16	`NAMA_FILE_CSV = ["hospital_data_analysis.csv", "hospital data analysis.csv"]`	Daftar nama file CSV yang dicari: memakai garis bawah atau memakai spasi.
17	`JUMLAH_DITAMPILKAN = 5`	Konstanta jumlah rekam medis yang ditampilkan (5).

main.py — baris 53–63

```python
def buat_rekam_medis(baris):
    """[CLASS & OBJECT] Membuat object RekamMedis yang berisi object Pasien (komposisi)."""
    pasien = buat_pasien(baris)
    return RekamMedis(
        pasien,
        baris["Condition"],
        baris["Procedure"],
        float(baris["Cost"]),
        int(baris["Length_of_Stay"]),
        baris["Outcome"],
    )
```

Fungsi tiap baris · main.py baris 53–63

Baris	Kode	Fungsinya
53	`def buat_rekam_medis(baris):`	Fungsi `buat_rekam_medis()`: mengubah satu baris CSV menjadi object `RekamMedis`.
54	`"""[CLASS & OBJECT] Membuat object RekamMedis yang berisi object Pasien (komposisi)."""`	Docstring: keterangan fungsi.
55	`pasien = buat_pasien(baris)`	Membuat object pasien terlebih dahulu dari baris yang sama.
56	`return RekamMedis(`	Membuat object `RekamMedis` dengan isian berikut:
57	`pasien,`	object pasien (komposisi),
58	`baris["Condition"],`	kondisi dari kolom Condition,
59	`baris["Procedure"],`	prosedur dari kolom Procedure,
60	`float(baris["Cost"]),`	biaya dari kolom Cost, diubah dari teks menjadi angka desimal dengan `float()`,
61	`int(baris["Length_of_Stay"]),`	lama rawat dari kolom Length_of_Stay, diubah menjadi bilangan bulat dengan `int()`,
62	`baris["Outcome"],`	hasil perawatan dari kolom Outcome.
63	`)`	Penutup pemanggilan `RekamMedis(...)`.

main.py — baris 66–81

```python
def muat_data(lokasi_file):
    """Membaca CSV, membuat object tiap baris, dan mencatat baris yang tidak valid."""
    daftar_rekam = []
    baris_dilewati = []  # nomor baris CSV yang tidak valid
    # utf-8-sig: tetap aman jika CSV pernah disimpan ulang lewat Excel (ada BOM)
    with open(lokasi_file, mode="r", encoding="utf-8-sig", newline="") as file:
        # restval="": kolom yang hilang dianggap teks kosong, jadi ikut ditolak validasi
        pembaca = csv.DictReader(file, restval="")
        for baris in pembaca:
            try:
                daftar_rekam.append(buat_rekam_medis(baris))
            except ValueError as error:
                # Baris yang melanggar aturan dilewati dan dicatat, program tetap berjalan
                baris_dilewati.append(pembaca.line_num)
                print(f"[PERINGATAN] Baris {pembaca.line_num} dilewati: {error}")
    return daftar_rekam, baris_dilewati
```

Fungsi tiap baris · main.py baris 66–81

Baris	Kode	Fungsinya
66	`def muat_data(lokasi_file):`	Fungsi `muat_data()`: membaca seluruh file CSV.
67	`"""Membaca CSV, membuat object tiap baris, dan mencatat baris yang tidak valid."""`	Docstring: keterangan fungsi.
68	`daftar_rekam = []`	Menyiapkan list kosong untuk menampung semua object.
69	`baris_dilewati = []  # nomor baris CSV yang tidak valid`	Menyiapkan list `baris_dilewati` untuk mencatat nomor baris CSV yang tidak valid.
70	`# utf-8-sig: tetap aman jika CSV pernah disimpan ulang lewat Excel (ada BOM)`	Komentar: alasan memakai encoding utf-8-sig.
71	`with open(lokasi_file, mode="r", encoding="utf-8-sig", newline="") as file:`	Membuka file CSV; `with` memastikan file ditutup otomatis, dan utf-8-sig aman untuk file yang disimpan lewat Excel.

Baris	Kode	Fungsinya
72	`# restval="": kolom yang hilang dianggap teks kosong, jadi ikut ditolak validasi`	Komentar: alasan memakai `restval=""`.
73	`pembaca = csv.DictReader(file, restval="")`	`csv.DictReader` membaca setiap baris sebagai dictionary berdasarkan nama kolom, misalnya `baris["Age"]`. `restval=""` mengisi kolom yang hilang dengan teks kosong, sehingga baris yang kolomnya kurang ditolak validasi, bukan menghentikan program.
74	`for baris in pembaca:`	Mengulang setiap baris data di file.
75	`try:`	`try`: mencoba membuat object dari baris ini.
76	`daftar_rekam.append(buat_rekam_medis(baris))`	Membuat object rekam medis lalu menambahkannya ke list dengan `append`.
77	`except ValueError as error:`	`except ValueError`: menangkap error dari setter (atau dari `int()`/`float()`) jika data melanggar aturan.
78	`# Baris yang melanggar aturan dilewati dan dicatat, program tetap berjalan`	Komentar: baris yang salah dilewati dan dicatat, program tetap berjalan.
79	`baris_dilewati.append(pembaca.line_num)`	Mencatat nomor baris yang sedang dibaca (`pembaca.line_num`) ke list `baris_dilewati`.
80	`print(f"[PERINGATAN] Baris {pembaca.line_num} dilewati: {error}")`	Mencetak peringatan beserta nomor baris dan alasannya.
81	`return daftar_rekam, baris_dilewati`	Mengembalikan dua hasil: list object yang valid dan list nomor baris yang dilewati.

main.py — baris 221–240

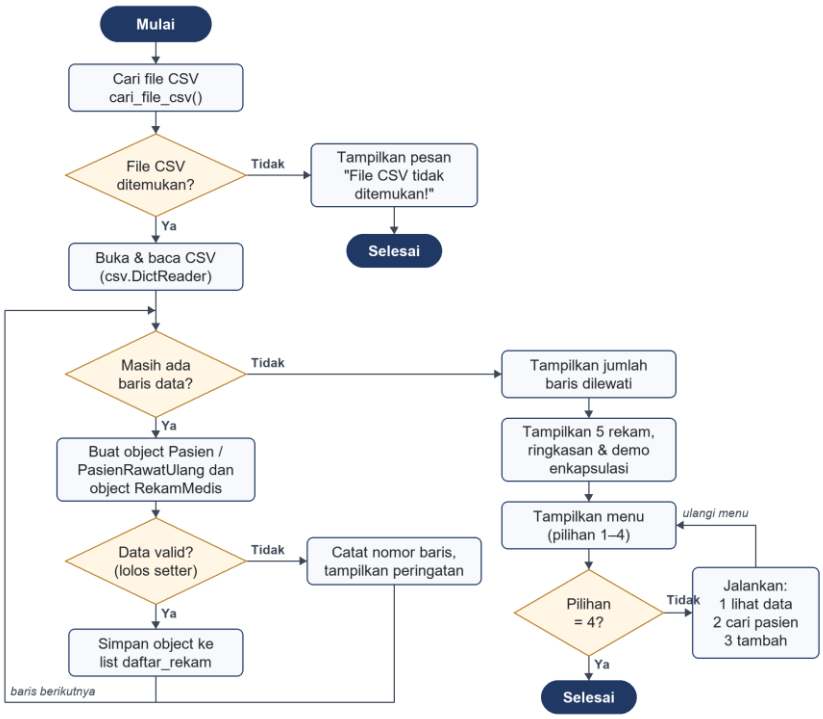
```
221 def jalankan_menu(daftar_rekam):
222     """Menampilkan menu berulang-ulang sampai pengguna memilih Keluar."""
223     while True:
224         cetak_judul("MENU")
225         print(f"1. Lihat {JUMLAH_DATA_MENU} data pertama")
226         print("2. Cari pasien berdasarkan Patient_ID")
227         print("3. Tambah pasien baru")
228         print("4. Keluar")
229         pilihan = input("Pilih menu (1-4): ").strip()
230         if pilihan == "1":
231             menu_lihat_data(daftar_rekam)
232         elif pilihan == "2":
233             menu_cari_pasien(daftar_rekam)
234         elif pilihan == "3":
235             menu_tambah_pasien(daftar_rekam)
236         elif pilihan == "4":
237             print("Terima kasih. Program selesai.")
238             break
239         else:
240             print("Pilihan tidak tersedia. Masukkan angka 1 sampai 4.")
```

Fungsi tiap baris · main.py baris 221–240

Baris	Kode	Fungsinya
221	`def jalankan_menu(daftar_rekam):`	Fungsi `jalankan_menu()`: menampilkan menu interaktif.
222	`"""Menampilkan menu berulang-ulang sampai pengguna memilih Keluar."""`	Docstring: keterangan fungsi.
223	`while True:`	`while True` membuat perulangan tanpa batas; menu tampil lagi setelah setiap pilihan selesai.
224	`cetak_judul("MENU")`	Mencetak judul MENU.
225	`print(f"1. Lihat {JUMLAH_DATA_MENU} data pertama")`	Mencetak pilihan 1 (jumlah datanya diambil dari konstanta `JUMLAH_DATA_MENU`).
226	`print("2. Cari pasien berdasarkan Patient_ID")`	Mencetak pilihan 2.
227	`print("3. Tambah pasien baru")`	Mencetak pilihan 3.
228	`print("4. Keluar")`	Mencetak pilihan 4.
229	`pilihan = input("Pilih menu (1-4): ").strip()`	Membaca pilihan pengguna dengan `input()`; `strip()` membuang spasi yang tidak sengaja terketik.
230	`if pilihan == "1":`	Jika pilihan "1" …

Baris	Kode	Fungsinya
231	`menu_lihat_data(daftar_rekam)`	… menjalankan `menu_lihat_data()`.
232	`elif pilihan == "2":`	Jika pilihan "2" …
233	`menu_cari_pasien(daftar_rekam)`	… menjalankan `menu_cari_pasien()`.
234	`elif pilihan == "3":`	Jika pilihan "3" …
235	`menu_tambah_pasien(daftar_rekam)`	… menjalankan `menu_tambah_pasien()`.
236	`elif pilihan == "4":`	Jika pilihan "4" …
237	`print("Terima kasih. Program selesai.")`	… mencetak salam penutup …
238	`break`	… lalu `break` menghentikan perulangan `while`, sehingga program selesai.
239	`else:`	Selain 1 sampai 4 …
240	`print("Pilihan tidak tersedia. Masukkan angka 1 sampai 4.")`	… mencetak pesan pilihan tidak tersedia, lalu menu tampil lagi.

Flowchart program utama



Alur program utama (main.py)

Naskah Penyampaian

Sama dengan **Naskah Presentasi Kelompok 3.pdf** dan catatan pembicara PPT. Versi lengkap dengan tanda KLIK dan tunjuk ada di dokumen naskah.

Slide 27 *Terima kasih, Revalinda. Saya Andi, akan menjelaskan program utama di main.py, termasuk menu interaktifnya.*

Slide 28 *Semua class tadi dijalankan dari main.py. Program mencari file CSV lebih dulu; kalau tidak ada, program berhenti dengan pesan. Kalau ada, setiap baris diubah menjadi object lalu dicek. Baris yang valid disimpan ke list, yang tidak valid dilewati dan dicatat nomornya. Setelah itu program menampilkan output awal, lalu masuk ke menu yang berulang sampai pengguna memilih keluar. Setiap langkah punya fungsinya sendiri: cari_file_csv mencari file, muat_data membaca CSV, buat_pasien dan buat_rekam_medis membuat object, tiga fungsi berikutnya menampilkan output awal, dan jalankan_menu menjalankan menu. Semuanya dipanggil berurutan oleh fungsi main, yang hanya berjalan jika file ini dijalankan langsung, lewat pengecekan if __name__ di baris paling bawah.*

Slide 29 *Sekarang isi fungsi muat_data. Baris 69 menyiapkan list baris_dilewati untuk mencatat nomor baris yang tidak valid. Baris 71 membuka file CSV; with menutup file otomatis, dan utf-8-sig aman untuk file yang pernah disimpan lewat Excel. Baris 73 membuat DictReader, yang membaca setiap baris sebagai dictionary berdasarkan nama kolom. restval diisi teks kosong, jadi baris yang kolomnya kurang ikut ditolak validasi, bukan membuat program berhenti. Baris 74 mengulang setiap baris, baris 75 mencoba memprosesnya, dan baris 76 menambahkan object ke list. Kalau data melanggar aturan, baris 77 menangkap ValueError, baris 79 mencatat nomor barisnya, dan baris 80 mencetak peringatan, lalu program tetap berjalan. Baris 81 mengembalikan list object dan list baris yang dilewati. Kami sudah mengujinya dengan baris rusak, dan untuk data asli kami, baris yang dilewati nol.*

Slide 30 *Bagian terakhir adalah menu interaktif di fungsi jalankan_menu. Baris 223, while True, membuat perulangan, jadi menu muncul lagi setiap kali satu pilihan selesai. Baris 224 sampai 228 mencetak judul dan empat pilihan. Baris 229 membaca pilihan dengan input. Lalu if dan elif menjalankan fungsinya: pilihan 1 menu_lihat_data, pilihan 2 menu_cari_pasien, dan pilihan 3 menu_tambah_pasien. Pilihan 4 mencetak salam penutup, lalu break di baris 238 menghentikan perulangan. Pilihan lain cukup dijawab dengan pesan, bukan error. Selanjutnya, Alfira akan menunjukkan hasil output program dan demo menunya.*

Persiapan Tanya Jawab

T **Kenapa memakai csv.DictReader, bukan csv.reader?**

J DictReader membaca baris berdasarkan nama kolom (baris["Age"]), sehingga kode lebih mudah dibaca dan tidak bergantung pada urutan kolom.

T **Apa yang terjadi jika ada baris CSV yang kolomnya kurang?**

J Kolom yang hilang diisi teks kosong karena restval="", lalu ditolak validasi dengan ValueError. Baris itu dilewati dan dicatat, program tetap berjalan.

T **Kenapa menu memakai while True?**

J Supaya menu tampil lagi setiap kali satu pilihan selesai. Perulangan baru berhenti saat pengguna memilih 4, karena ada perintah break.

T **Apa yang terjadi jika file CSV tidak ada?**

J Program menampilkan pesan “File CSV tidak ditemukan!” lalu berhenti dengan rapi, tanpa error panjang.

T **Kenapa memakai encoding utf-8-sig?**

J Supaya file tetap terbaca benar walaupun pernah disimpan ulang lewat Excel, yang kadang menambahkan karakter BOM di awal file.

BAGIAN 9 | ALFIRA YUNITA SYAM

Hasil Output, Demo Menu & Kesimpulan

Durasi: ± 4 menit • **Konsep:** Semua konsep • **Fokus:** Output python main.py & menu

Poin yang Dijelaskan

- Menjalankan program dengan `python main.py` dan menunjukkan output-nya: 984 object dimuat dan **0 baris dilewati**.
- Hanya **5 rekam medis pertama** yang ditampilkan (bukan 984 baris) dengan *slicing* `daftar_rekam[:JUMLAH_DITAMPILKAN]` (konstanta bernilai 5). Label class terlihat jelas: `Pasien` dan `PasienRawatUlang`.
- Ringkasan data asli: **984 pasien**; **264 pasien rawat ulang (26.83%)**; rata-rata biaya **8,367.48**; rata-rata lama rawat **37.66 hari**; kondisi paling sering **Fractured Leg dan Heart Attack** (masing-masing 67 pasien).
- Kondisi terbanyak **seri**, jadi program sengaja menampilkan keduanya (kode baris 113–123). Kalau hanya menampilkan satu, hasilnya menyesatkan.
- Cara menghitung ringkasan: perulangan pada list, `isinstance()` untuk menghitung pasien rawat ulang, dan *dictionary* untuk menghitung jumlah tiap kondisi.
- Demo enkapsulasi membuktikan setter menolak data yang tidak valid.
- **Demo menu**: pilihan 1 menampilkan 10 data pertama; pilihan 2 mencari Patient_ID 2 (ditemukan sebagai `PasienRawatUlang` dengan baris Status Rawat) dan ID 60 (memang tidak ada di data, jadi tidak ditemukan); pilihan 3 menambah pasien dengan ID otomatis 1001, sedangkan kepuasan 9 ditolak setter.
- Pasien baru disimpan di list selama program berjalan; file CSV asli tidak diubah.
- **Kesimpulan**: ketiga konsep OOP berhasil diterapkan pada data nyata. Lalu menutup presentasi dan membuka sesi tanya jawab.

Kode & Tampilan yang Ditunjukkan

main.py — baris 113–123

```
113     # Kondisi terbanyak bisa lebih dari satu jika jumlahnya sama (seri)
114     jumlah_terbanyak = max(jumlah_per_kondisi.values())
115     kondisi_terbanyak = []
116     for kondisi, jumlah in jumlah_per_kondisi.items():
117         if jumlah == jumlah_terbanyak:
118             kondisi_terbanyak.append(kondisi)
119
120     if len(kondisi_terbanyak) > 1:
121         keterangan = f"masing-masing {jumlah_terbanyak} pasien"
122     else:
123         keterangan = f"{jumlah_terbanyak} pasien"
```

Fungsi tiap baris · main.py baris 113–123

Baris	Kode	Fungsinya
113	`# Kondisi terbanyak bisa lebih dari satu jika jumlahnya sama (seri)`	Komentar: kondisi terbanyak bisa lebih dari satu (seri).
114	`jumlah_terbanyak = max(jumlah_per_kondisi.values())`	`max` mencari jumlah pasien terbanyak dari semua kondisi.
115	`kondisi_terbanyak = []`	Menyiapkan list kosong untuk kondisi teratas.
116	`for kondisi, jumlah in jumlah_per_kondisi.items():`	Mengulang setiap pasangan kondisi dan jumlahnya.
117	`if jumlah == jumlah_terbanyak:`	Jika jumlahnya sama dengan jumlah terbanyak …
118	`kondisi_terbanyak.append(kondisi)`	… kondisi itu dimasukkan ke list, sehingga kondisi yang seri tidak hilang.

Baris	Kode	Fungsinya
120	`if len(kondisi_terbanyak) > 1:`	Jika kondisi teratas lebih dari satu …
121	`keterangan = f"masing-masing {jumlah_terbanyak} pasien"`	… keterangannya “masing-masing 67 pasien”.
122	`else:`	Jika hanya satu …
123	`keterangan = f"{jumlah_terbanyak} pasien"`	… keterangannya cukup “67 pasien”.

Output ringkasan data

```
============================================================
RINGKASAN DATA
============================================================
Total pasien              : 984
Jumlah pasien rawat ulang : 264 (26.83%)
Rata-rata biaya           : 8,367.48
Rata-rata lama rawat      : 37.66 hari
Kondisi paling sering     : Fractured Leg, Heart Attack (masing-masing 67 pasien)
```

main.py — baris 201–218

```python
def menu_tambah_pasien(daftar_rekam):
    """Menu 3: menambah pasien baru ke list (file CSV asli tidak diubah)."""
    cetak_judul("TAMBAH PASIEN BARU")
    # Patient_ID dibuat otomatis (ID terbesar + 1) agar tidak ada ID ganda
    id_baru = max(rekam.pasien.patient_id for rekam in daftar_rekam) + 1
    print(f"Patient_ID baru (otomatis)  : {id_baru}")
    baris = {"Patient_ID": str(id_baru)}
    for kolom, pertanyaan in PERTANYAAN_PASIEN_BARU:
        baris[kolom] = input(f"{pertanyaan:<28}: ").strip()
    try:
        # [ENKAPSULASI] Input diperiksa oleh setter yang sama dengan data CSV
        rekam_baru = buat_rekam_medis(baris)
    except ValueError as error:
        print(f"Data tidak valid: {error}")
        print("Pasien baru tidak ditambahkan.")
        return
    daftar_rekam.append(rekam_baru)
    print(f"Pasien ID {id_baru} berhasil ditambahkan. Jumlah data sekarang:
{len(daftar_rekam)}")
```

Fungsi tiap baris · main.py baris 201–218

Baris	Kode	Fungsinya
201	`def menu_tambah_pasien(daftar_rekam):`	Fungsi `menu_tambah_pasien()` untuk menu 3.
202	`"""Menu 3: menambah pasien baru ke list (file CSV asli tidak diubah)."""`	Docstring: pasien baru disimpan ke list; file CSV asli tidak diubah.
203	`cetak_judul("TAMBAH PASIEN BARU")`	Mencetak judul menu.
204	`# Patient_ID dibuat otomatis (ID terbesar + 1) agar tidak ada ID ganda`	Komentar: cara membuat Patient_ID baru.
205	`id_baru = max(rekam.pasien.patient_id for rekam in daftar_rekam) + 1`	`max()` mencari Patient_ID terbesar dari semua rekam, lalu ditambah 1 (data asli: 1000 → 1001), sehingga tidak ada ID ganda.
206	`print(f"Patient_ID baru (otomatis)  : {id_baru}")`	Menampilkan Patient_ID baru kepada pengguna.
207	`baris = {"Patient_ID": str(id_baru)}`	Menyiapkan dictionary `baris` berbentuk sama seperti satu baris CSV, diawali Patient_ID.
208	`for kolom, pertanyaan in PERTANYAAN_PASIEN_BARU:`	Mengulang setiap pasangan (kolom, pertanyaan) dari `PERTANYAAN_PASIEN_BARU`.
209	`baris[kolom] = input(f"{pertanyaan:<28}: ").strip()`	Menampilkan pertanyaan, membaca jawaban dengan `input()`, membuang spasi dengan `strip()`, lalu menyimpannya di kolom yang sesuai.
210	`try:`	`try`: mencoba membuat object dari jawaban pengguna.
211	`# [ENKAPSULASI] Input diperiksa oleh setter yang sama dengan data CSV`	Komentar: input diperiksa oleh setter yang sama dengan data CSV (enkapsulasi).

Baris	Kode	Fungsinya
212	`rekam_baru = buat_rekam_medis(baris)`	Memanggil `buat_rekam_medis()`, fungsi yang juga dipakai untuk setiap baris CSV, sehingga semua validasi setter berlaku.
213	`except ValueError as error:`	`except ValueError`: menangkap data yang tidak valid, misalnya kepuasan 9.
214	`print(f"Data tidak valid: {error}")`	Mencetak alasan data ditolak.
215	`print("Pasien baru tidak ditambahkan.")`	Mencetak bahwa pasien tidak ditambahkan.
216	`return`	`return` kembali ke menu tanpa menambah data.
217	`daftar_rekam.append(rekam_baru)`	Jika valid, object rekam medis baru ditambahkan ke list dengan append.
218	`print(f"Pasien ID {id_baru} berhasil ditambahkan. Jumlah data sekarang: {len(daftar_rekam)}")`	Mencetak pesan berhasil beserta jumlah data sekarang.

Output menu 1: lihat 10 data pertama

```
Pilih menu (1-4): 1
============================================================
10 DATA PERTAMA (dari 984 data)
============================================================
ID   Umur  Gender  Kondisi                    Biaya  Rawat Ulang
1    45    Female  Heart Disease          15,000.00  Tidak
2    60    Male    Diabetes                2,000.00  Ya
3    32    Female  Fractured Arm             500.00  Tidak
4    75    Male    Stroke                 10,000.00  Ya
5    50    Female  Cancer                 25,000.00  Tidak
6    68    Male    Hypertension            1,000.00  Tidak
7    55    Female  Appendicitis            8,000.00  Tidak
8    40    Male    Fractured Leg           3,000.00  Tidak
9    70    Female  Heart Attack           18,000.00  Ya
10   25    Male    Allergic Reaction         100.00  Tidak
```

Output menu 2: cari Patient_ID 2

```
Pilih menu (1-4): 2
============================================================
CARI PASIEN BERDASARKAN PATIENT_ID
============================================================
Masukkan Patient_ID: 2
Pasien ditemukan (object dari class: PasienRawatUlang)
--- Data Pasien ---
ID Pasien      : 2
Umur           : 60 tahun
Jenis Kelamin  : Male
Kepuasan       : 3/5
Status Rawat   : Pasien rawat ulang
--- Data Medis ---
Kondisi        : Diabetes
Prosedur       : Insulin Therapy
Biaya          : 2,000.00
Lama Rawat     : 3 hari
Biaya per Hari : 666.67
Hasil          : Stable
```

Output menu 3: tambah pasien baru (valid)

```
Pilih menu (1-4): 3
============================================================
TAMBAH PASIEN BARU
============================================================
Patient_ID baru (otomatis)  : 1001
Umur                        : 34
Jenis kelamin (Male/Female) : Female
Kondisi                     : Fractured Arm
Prosedur                    : X-Ray and Splint
Biaya                       : 500
Lama rawat (hari)           : 2
Rawat ulang (Yes/No)        : No
Hasil (Recovered/Stable)    : Recovered
Kepuasan (1-5)              : 5
Pasien ID 1001 berhasil ditambahkan. Jumlah data sekarang: 985
```

Output menu 3: kepuasan 9 ditolak setter

```
Pilih menu (1-4): 3
============================================================
TAMBAH PASIEN BARU
============================================================
Patient_ID baru (otomatis)  : 1002
Umur                        : 52
Jenis kelamin (Male/Female) : Male
Kondisi                     : Hypertension
Prosedur                    : Medication and Counseling
Biaya                       : 1000
Lama rawat (hari)           : 3
Rawat ulang (Yes/No)        : No
Hasil (Recovered/Stable)    : Stable
Kepuasan (1-5)              : 9
Data tidak valid: Kepuasan harus bilangan bulat 1 sampai 5, bukan 9
Pasien baru tidak ditambahkan.
```

Naskah Penyampaian

Sama dengan **Naskah Presentasi Kelompok 3.pdf** dan catatan pembicara PPT. Versi lengkap dengan tanda KLIK dan tunjuk ada di dokumen naskah.

Slide 31 *Terima kasih, Andi. Saya Alfira, akan menunjukkan hasil program, demo menu, dan kesimpulan.*

Slide 32 *Ini hasil saat program dijalankan. Program memuat 984 object, dan baris yang dilewati nol, artinya semua data asli valid. Agar ringkas, hanya lima rekam medis pertama yang ditampilkan: di fungsi tampilkan_rekam_pertama, daftar_rekam[:JUMLAH_DITAMPILKAN] mengambil lima object pertama, dan type(rekam.pasien).__name__ mengambil nama class-nya, sehingga terlihat rekam kedua dan keempat adalah PasienRawatUlang. Seluruh 984 data tetap dipakai untuk menghitung ringkasan.*

Slide 33 *Totalnya ada 984 pasien. Sebanyak 264 pasien dirawat ulang, sekitar dua puluh enam koma delapan tiga persen. Rata-rata biayanya delapan ribu tiga ratus enam puluh tujuh koma empat delapan, dan rata-rata lama rawatnya tiga puluh tujuh koma enam enam hari. Di fungsi tampilkan_ringkasan, perulangan for membaca setiap rekam, isinstance menghitung pasien rawat ulang, total biaya dan lama rawat dibagi jumlah pasien, dan jumlah tiap kondisi dihitung dengan dictionary. Kondisi yang paling sering muncul ternyata ada dua.*

Slide 34 *Fractured Leg dan Heart Attack sama-sama dialami 67 pasien, jadi hasilnya seri. Sebaran kelima belas kondisi juga hampir merata, antara 64 sampai 67 pasien. Di kode, baris 114 memakai max untuk mencari jumlah terbanyak, lalu baris 116 sampai 118 mengumpulkan semua kondisi yang jumlahnya sama dengan nilai itu. Karena itu kondisi yang seri tidak hilang; kalau hanya satu yang ditampilkan, hasilnya menyesatkan.*

Slide 35 *Sekarang demo menunya, dengan output asli program. Pilihan 1 menampilkan sepuluh data pertama dalam tabel singkat. Pilihan 2 mencari Patient_ID 2: fungsi cari_rekam memeriksa setiap rekam, dan pasien ditemukan sebagai PasienRawatUlang, lengkap dengan baris Status Rawat. ID 60, yang memang tidak ada di data, dijawab tidak ditemukan, dan input huruf ditolak oleh try-except. Pilihan 3 menambah pasien. Baris 205 membuat ID otomatis, yaitu ID terbesar ditambah satu, jadi 1001, lalu baris 212 memanggil buat_rekam_medis, sehingga data baru diperiksa setter yang sama dengan data CSV. Buktinya, kepuasan 9 ditolak. Pasien yang valid disimpan ke list di baris 217, sedangkan file CSV asli tidak diubah.*

Slide 36 *Sebagai kesimpulan, ketiga konsep OOP berhasil kami terapkan pada data nyata. Class dan object: empat class di file terpisah, dan 984 baris CSV menjadi 1968 object. Enkapsulasi: semua atribut private dan divalidasi setter, termasuk data dari menu. Inheritance: Orang ke Pasien ke PasienRawatUlang, dengan super, overriding, dan method khusus status_rawat. Ke depan, program bisa dikembangkan dengan fitur ubah dan hapus data, atau penyimpanan ke database.*

Slide 37 *Sekian presentasi dari Kelompok 3. Terima kasih atas perhatian Bapak/Ibu dosen dan teman-teman. Kami persilakan jika ada pertanyaan.*

Persiapan Tanya Jawab

T **Kenapa kondisi paling sering ada dua?**

J Karena Fractured Leg dan Heart Attack sama-sama muncul 67 kali (seri). Program sengaja menampilkan semua kondisi dengan jumlah tertinggi agar hasilnya jujur.

T **Kenapa rata-rata lama rawat cukup tinggi (37.66 hari)?**

J Itu hasil dari data asli. Lama rawat di dataset berkisar 1 sampai 76 hari, dan program hanya menghitung rata-ratanya.

T **Kenapa hanya 5 rekam medis yang ditampilkan?**

J Supaya output tetap ringkas dan mudah dibaca. Seluruh 984 data tetap dimuat dan dipakai untuk menghitung ringkasan.

T **Apakah pasien yang ditambah lewat menu tersimpan ke file CSV?**

J Tidak. Pasien baru disimpan di list selama program berjalan, jadi file CSV asli tetap utuh. Penyimpanan permanen bisa menjadi pengembangan berikutnya.

T **Kenapa ID pasien baru 1001?**

J ID dibuat otomatis dari ID terbesar ditambah satu. ID terbesar di data adalah 1000, jadi pasien baru mendapat 1001 dan tidak ada ID ganda.

T **Bagaimana jika diminta demo langsung?**

J Jalankan python main.py, lalu ketik masukan yang sama dengan slide demo: 1; 2 lalu 2; 2 lalu 60; 3 lalu 34, Female, Fractured Arm, X-Ray and Splint, 500, 2, No, Recovered, 5; 2 lalu 1001; 4. Hasilnya akan sama dengan output di slide.

PENUTUP & PERSIAPAN DEMO

Urutan Estafet Pembicara

1. Riswan → 2. Yus Aidil → 3. Fahmi → 4. Diva Nurislamia Agfa → 5. Rosa Ramadani → 6. Sofiang → 7. Revalinda → 8. Andi Rezky Nabila → 9. Alfira Yunita Syam

Checklist Sebelum Presentasi

- ☐ Python 3 sudah terpasang. Cek dengan perintah `python --version`.
- ☐ File `hospital data analysis.csv` berada **satu folder** dengan `main.py`.
- ☐ Jalankan `python main.py` sekali sebelum maju untuk memastikan output muncul tanpa error.
- ☐ Coba menu: pilih 2 lalu ketik ID 2 dan 60, pilih 3 untuk menambah pasien, lalu pilih 4 untuk keluar.
- ☐ Perbesar ukuran huruf terminal dan editor agar terbaca jelas di proyektor.
- ☐ Buka file kode sesuai urutan pembicara: `orang.py` → `pasien.py` → `pasien_rawat_ulang.py` → `rekam_medis.py` → `main.py`.
- ☐ Setiap pembicara hafal minimal satu jawaban dari bagian **Persiapan Tanya Jawab** miliknya.

Letak Konsep OOP di Dalam Kode

Konsep	Letak di kode	Dijelaskan oleh
Class & Object	4 class: orang.py baris 6, pasien.py baris 9, pasien_rawat_ulang.py baris 10, rekam_medis.py baris 9. Object dibuat di main.py baris 46, 48, 56 dan disimpan ke list (baris 76). Komposisi: rekam_medis.py baris 14. Menu interaktif: main.py baris 221–240.	Fahmi, Revalinda, Andi
Enkapsulasi	Atribut private + @property/@setter di orang.py, pasien.py, rekam_medis.py. Validasi raise ValueError: orang.py baris 24 & 35, pasien.py baris 38, rekam_medis.py baris 41, 52, 63, 74 & 85, main.py baris 50. Demo try-except: main.py baris 139–143. Input menu tambah pasien diperiksa setter yang sama (main.py baris 212).	Diva, Revalinda
Inheritance	Orang → Pasien → PasienRawatUlang. super().__init__(): pasien.py baris 14 & pasien_rawat_ulang.py baris 15. Method khusus status_rawat(): pasien_rawat_ulang.py baris 18. Override tampilkan_info(): orang.py baris 39 → pasien.py baris 42 → pasien_rawat_ulang.py baris 23. Polimorfisme: rekam_medis.py baris 98.	Rosa, Sofiang

Kesimpulan Singkat

- **Class & Object**: 4 class di file terpisah; 984 baris CSV diubah menjadi 1968 object yang disimpan di list.
- **Enkapsulasi**: semua atribut private (__), diakses lewat `@property`/setter, dan divalidasi dengan `ValueError`.
- **Inheritance**: `Orang` → `Pasien` → `PasienRawatUlang`, dengan `super().__init__()`, override `tampilkan_info()`, dan method khusus `status_rawat()`.